

 **DTU Compute**
Department of Applied Mathematics and Computer Science

Attacks on Personalized Federated Learning

An Exploratory Analysis

Aatharva Ravindra Kawade (s234681)

Kongens Lyngby 2026



DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Matematiktorvet
Building 303B
2800 Kongens Lyngby, Denmark
Phone +45 4525 3031
compute@compute.dtu.dk
www.compute.dtu.dk

Abstract

Federated learning is commonly presented as a privacy-aware machine learning paradigm because raw training data remain on local clients and only model updates are communicated to a central server. However, prior work has shown that gradients and parameter updates can still leak sensitive information through gradient inversion attacks. This thesis investigates how personalization strategies in federated learning influence the effectiveness of such attacks compared to standard non-personalized federated learning. The work is conducted as a controlled exploratory analysis using FedAvg as the baseline, FedPer as a layer-based personalized federated learning method, and Per-FedAvg as a meta-learning-based personalized federated learning method. Two gradient inversion attacks, Deep Leakage from Gradients and Inverting Gradients, are evaluated on CIFAR-10 using a LeNet-style convolutional neural network under a single sample, highly non-IID client setting. Reconstruction quality is measured using MSE, PSNR, SSIM, and label recovery accuracy. The results show that FedAvg is highly vulnerable under the tested conditions, with both attacks recovering labels perfectly and Inverting Gradients producing the strongest image reconstructions. FedPer reduces image reconstruction quality as more personalization layers are kept private, but label leakage remains present, showing that partial model sharing does not eliminate semantic information exposure. Per-FedAvg produces the strongest separation between image and label leakage: visually meaningful image reconstruction is largely prevented, while Inverting Gradients still recovers labels with near-perfect accuracy. These findings indicate that personalization changes the structure and form of leakage rather than removing it entirely. In the tested setting, personalized federated learning reduces pixel-level reconstruction risk compared to FedAvg, but label leakage remains a significant privacy concern. The results should be interpreted as an exploratory baseline rather than a complete privacy audit, since the experiment uses fixed attack hyperparameters, a simplified Per-FedAvg implementation, single sample updates, and a limited model architecture.

Preface

This Bachelor's thesis was prepared at the department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a bachelor's degree in science.

Kongens Lyngby, May 16, 2026

A handwritten signature in black ink, reading 'A. Kawade'. The signature is stylized with a large loop at the beginning and a long horizontal stroke at the end.

Aatharva Ravindra Kawade (s234681)

Acknowledgments

I would like to express my sincere gratitude to my thesis supervisor, Hiba Nassar, for her guidance, support, and encouragement throughout the process of writing this bachelor's thesis. Her knowledge, kindness, and constructive feedback have been highly valuable during the development of this work, and her supervision has helped guide both the academic direction and the overall thesis process.

I would also like to acknowledge the use of artificial intelligence tools during the preparation of this thesis. These tools were used as a writing aid, primarily to support rephrasing, language refinement, and the improvement of textual clarity. The academic content, analysis, interpretations, and final responsibility for the thesis remain my own.

Contents

Abstract	i
Preface	ii
Acknowledgments	iii
Contents	iv
1 Introduction	1
1.1 Related Work	3
2 Theory	5
2.1 Federated learning	5
2.2 Personalized Federated learning	6
2.3 Gradient Inversion Attacks	11
2.4 Convolutional Neural Networks	14
2.5 Performance Metrics	16
3 Method	18
3.1 Research Objective and Experimental Overview	18
3.2 Algorithm Design	19
3.3 Attack Design	20
3.4 Data	21
3.5 Model	22
3.6 Evaluation Metrics	22
3.7 Numerical Experiments	23
4 Results	28
5 Discussion	33
6 Conclusion	41
A Appendix	42
Bibliography	47

Introduction

The use of artificial intelligence (AI) has expanded rapidly in recent years. No longer confined to a limited number of specialized domains, AI technologies are now embedded across a wide range of sectors, including healthcare, finance, technology, entertainment, and beyond. At the core of this expansion lies the data-driven predictive capability of AI systems, enabled by machine learning (ML). ML provides the mathematical and statistical framework through which models learn from data by identifying patterns and iteratively improving their predictive performance. In contrast to earlier AI approaches, where rules and behaviors were explicitly programmed, modern ML systems rely on algorithmic optimization grounded in statistical theory to detect patterns that minimize loss with respect to training data. Data, therefore, constitutes the foundation of ML; the performance and generalization capability of a model are inherently dependent on the quality and quantity of the data used during training. However, increasing regulatory constraints, such as the General Data Protection Regulation (GDPR), together with growing public awareness of data privacy and autonomy, have made large-scale centralized data collection progressively more challenging. Federated learning (FL) has emerged as a potential solution to this tension between data utility and privacy.

Federated learning is a distributed machine learning paradigm in which model training is conducted collaboratively across multiple participating clients. Rather than aggregating raw data in a centralized repository, FL retains data on the client devices where it is originally generated or located. In a typical centralized FL architecture, each client performs local model updates using its private dataset and transmits only the resulting gradients or parameter updates to a central server. The server then aggregates these updates to produce a revised set of global model weights, thereby maintaining a unified global model without direct access to client-level data. Because data remains decentralized across clients, data heterogeneity is an inherent challenge in FL settings. The local datasets often exhibit non-independent and identically distributed (non-IID) characteristics, meaning that the data distribution observed by one client may differ from the distributions observed by other clients or from the overall population distribution. This is problematic because each client performs local optimization with respect to its own local objective, which is not necessarily an unbiased or representative approximation of the global objective. As a result, the updates produced by different clients can become biased relative to the global loss and exhibit high variance across clients. This can cause the aggregated update to move in unstable or conflicting directions, leading to client drift, slower convergence, and degraded global model performance [Li+20b; Kar+20]. However, non-IID data does not always imply that personalization is required. If the training goal is still to learn a single predictor that performs well over the collective client data distribution, then non-IID data can also be addressed through non-personalized FL methods such as modified aggregation, regularization, proximal objectives, control variates, or client sampling strategies [Li+20a; Kar+20]. Personalized federated learning (PFL) becomes particularly relevant when the training goal is not only a strong global model, but a model that performs well for each client's own task or data distribution.

Personalized federated learning extends the foundational framework of federated learning to better accommodate settings in which model personalization is required. Its applicability is particularly significant in domains involving individualized tasks, such as personalized healthcare. As previously discussed, the primary objective of traditional FL is to learn a single global model over the collective client datasets. This objective is most appropriate when the client objectives are sufficiently aligned,

such that local updates provide useful information about the same underlying optimization problem. When the IID assumption is violated, this alignment becomes weaker because each client may optimize over a different local data distribution. In such cases, local optimization may no longer reliably reflect the global objective, since the gradients or parameter updates are shaped by client-specific label distributions or feature distributions. This can increase both the bias and variance of client updates relative to the global objective, which makes aggregation less stable and can slow convergence [Li+20b; Kar+20]. However, this issue should be separated from the broader personalization problem. Some non-IID settings still aim to learn one global predictor, and in such cases the purpose of the algorithm is to reduce the harmful optimization effects of heterogeneity. In contrast, when clients require models for distinct or personalized tasks, the objective itself changes. The goal is no longer only to learn a single unified model, but to develop models that achieve strong performance on each client’s specific task. For this reason, PFL has emerged as an area of significant research interest, particularly as modern applications increasingly prioritize user-specific optimization and tailored content. Several methodological approaches have been proposed within the PFL paradigm, including model fine-tuning, multitask learning, personalization layers, and meta-learning. More recent work has focused on meta-learning techniques due to their ability to rapidly adapt to new tasks. In this setting, the goal is to learn a model that serves as a strong initialization and effective adapter, which can then be further trained on client-specific data to produce a personalized model optimized for the client’s individual task.

Federated learning is inherently designed as a privacy-aware paradigm, as raw training data are not transmitted within the protocol. Nevertheless, prior studies [Bho+18; FJR15; Mel+19; ZLH19] have demonstrated that FL does not guarantee robust privacy protection. In particular, sensitive information about client data can be inferred from the gradients or parameter updates exchanged during communication with the central aggregator. Such threats fall under the category of inference attacks, and more specifically gradient inversion attacks (GIAs), in which an adversary reconstructs training inputs from the locally computed updates transmitted during communication rounds. These attacks are typically modeled under a semi-honest, or honest-but-curious, adversarial setting. The adversary adheres to the prescribed FL protocol and does not deviate from it to gain additional privileges, but instead exploits information leaked through legitimate gradient exchanges during the training phase. Moreover, many attack formulations assume that the adversarial participant is selected in multiple training rounds, thereby accumulating sufficient information to perform reconstruction. Notably, Zhu et al. [ZLH19] introduced the Deep Leakage from Gradients (DLG) method, demonstrating that both training inputs and corresponding labels can be recovered within only a few optimization iterations. Subsequently, Zhao et al. [ZMB20] proposed an improved variant, iDLG, which further refines the reconstruction process.

In this thesis, the privacy implications of personalized federated learning are investigated through an exploratory analysis of gradient inversion attacks. The work focuses on the interaction between personalization mechanisms and information leakage, with particular attention to whether the structural changes introduced by PFL alter the feasibility and effectiveness of reconstruction attacks. Per-FedAvg is used as the primary representative of meta-learning-based personalized federated learning, while FedAvg serves as the conventional federated learning baseline. In addition, FedPer is considered as a layer-partitioning-based personalization method in order to compare how different forms of personalization affect the information exposed during training. The central objective of this thesis is to examine how personalization changes the vulnerability of federated learning systems to traditional gradient inversion attacks, and whether these changes correspond to increased or decreased reconstruction risk under specific algorithmic conditions. This is studied by evaluating the reconstruction quality obtained from attacks applied to model updates generated under different federated learning algorithms. The analysis is conducted in a controlled experimental setting using image classification tasks, where client updates are produced using local non-IID data and then attacked using established gradient inversion methods. In particular, the thesis considers DLG and Inverting Gradients as representative traditional reconstruction attacks, as these methods directly exploit gradients or parameter updates to recover private training inputs.

1.1 Related Work

Federated learning was introduced by [McM+17] as a distributed training paradigm in which clients collaboratively train a shared model while keeping their raw data local. The Federated Averaging (FedAvg) algorithm proposed in that work remains the standard baseline for many FL studies, as it combines local stochastic gradient descent on client devices with server-side averaging of transmitted model updates. Subsequent surveys and position papers have identified statistical heterogeneity, systems heterogeneity, communication cost, and privacy leakage as central challenges in FL [Kai+21; Zha+21]. In particular, non-IID client data can cause local objectives to differ substantially from the global objective, thereby reducing the effectiveness of a single shared model. Methods such as FedProx address this issue by modifying the local optimization objective to improve robustness under heterogeneous client conditions [Li+20a]. However, such approaches still primarily aim to learn one global model, which may be insufficient when client-specific performance is the main objective.

Personalized federated learning (PFL) has emerged as a response to this limitation. Rather than enforcing a single model across all clients, PFL introduces mechanisms that allow client models to adapt to local data distributions while still benefiting from collaborative training. Existing PFL methods can broadly be grouped into approaches based on local fine-tuning, regularization, model interpolation, clustering, meta-learning, multitask learning, and parameter partitioning [Tan+23; Man+20]. Within this landscape, [FMO20] propose Per-FedAvg, which formulates personalization through the model-agnostic meta-learning framework. The objective is not simply to minimize the average global loss, but to learn an initialization from which each client can obtain a strong personalized model after one or a few local gradient steps. In contrast, [Ari+19] propose FedPer, which separates the model into shared base layers and private personalization layers. The base layers are trained collaboratively and communicated to the server, whereas the personalization layers remain local to each client. Other PFL methods, such as pFedMe, use regularized local objectives and Moreau envelopes to decouple personalized model optimization from global model learning [DTN20]. These methods demonstrate that personalization can be achieved through structurally different mechanisms, each of which changes what information is computed locally and what information is exposed during communication.

The security of FL has also been widely studied. [LYY20] distinguish between poisoning attacks, which target the integrity of the learned model, and inference attacks, which target the privacy of client data. Early work on privacy leakage showed that model outputs or updates may reveal sensitive information about training data [FJR15; Mel+19]. More directly, [ZLH19] introduced Deep Leakage from Gradients (DLG), showing that both inputs and labels can be reconstructed by optimizing dummy data so that its gradients match the observed gradients. [ZMB20] improved this approach with iDLG by showing that, for models trained with cross-entropy loss and one-hot labels, the ground-truth label can be inferred analytically from the signs of the gradients. [Gei+20] further improved reconstruction quality by replacing Euclidean gradient matching with a cosine similarity objective and incorporating image priors such as total variation regularization. Later evaluations show that the effectiveness of GIAs depends strongly on assumptions such as batch size, model architecture, initialization, label availability, and the number of local update steps [Hua+21].

A particularly relevant line of work studies whether gradient leakage remains practical under FedAvg rather than the simpler FedSGD setting. In FedSGD, the server observes gradients computed from a single local step, whereas in FedAvg, clients may perform several local updates before transmitting only the resulting model parameters. This makes reconstruction harder because the server does not observe the intermediate gradients or intermediate model states. [Dim+22] address this setting directly and show that data leakage can still occur in FedAvg, although the reconstruction problem becomes more difficult as batch size, number of local steps, and aggregation increase. Their results also motivate controlled experimental settings in which these factors are isolated.

Although gradient inversion attacks have been extensively studied for standard FL, their interaction

with personalized federated learning remains less explored. This is a significant gap because PFL changes the training objective, the local adaptation process, and in some algorithms the subset of parameters communicated to the server. Per-FedAvg introduces meta-learning updates that are explicitly optimized for client adaptation, while FedPer hides personalization layers but still communicates shared representation layers. These structural differences may change the amount, location, and form of information available to an adversary.

2.1 Federated learning

Federated learning is a distributed learning paradigm in which a collection of devices, referred to as clients or users, collaboratively train a shared global model while keeping their data localized. Each client performs training using its own private dataset, and only model updates are communicated to a central coordinating server for aggregation. This approach contrasts with traditional centralized machine learning, where all data are transferred to and stored in a central location prior to training. By avoiding direct data sharing, federated learning inherently promotes a privacy-preserving, or “privacy-by-design,” architecture.

In the commonly adopted centralized federated learning setting, a central server orchestrates the training process by aggregating model updates received from clients. Communication follows an iterative protocol in which the server distributes the current global model to a subset of clients, and clients return updated model parameters after performing local training [McM+17]. Importantly, only model parameters or gradients are exchanged, rather than raw data, which reduces both privacy risks and communication of sensitive information.

FedAvg Algorithm

[McM+17] introduced the Federated Averaging (FedAvg) algorithm, which is widely regarded as the foundational optimization method in federated learning. The algorithm combines local stochastic gradient descent (SGD) performed independently on a set of participating clients with a central server that aggregates the resulting local model updates.

FedAvg aims to solve the following global optimization problem:

$$\min_{w \in \mathbb{R}^d} f(w) := \frac{1}{n} \sum_{i=1}^n f_i(w) \quad (2.1)$$

where $f_i(w)$ denotes the local objective function of client i , typically defined as the empirical loss over its local dataset D_i , i.e., $f_i(w) = \frac{1}{|D_i|} \sum_{(x,y) \in D_i} l(x, y; w)$, with $l(x, y; w)$ being the loss of model parameters w on data sample (x, y) . Here, n denotes the number of participating clients. To facilitate optimization of the global objective, each client computes local updates by performing multiple steps of SGD on its own dataset. Concretely, starting from the current global model w^t , client i updates its local model by iterating:

$$w_i^{t+1} \leftarrow w_i^t - \eta \nabla f_i(w_i^t), \quad (2.2)$$

for several local steps, where η is the learning rate. These local updates approximate minimizing the client-specific objective $f_i(w)$.

At the beginning of training, the server initializes a global model and distributes it to a subset of clients. During each communication round, the selected clients perform local training using their private data and return their updated model parameters to the server. The server then aggregates these

local models, typically via a weighted average proportional to dataset sizes, to produce a new global model:

$$w^{t+1} = \sum_{i=1}^n \frac{|D_i|}{\sum_{j=1}^n |D_j|} w_i^{t+1}$$

This iterative process of local training and global aggregation is repeated until convergence. By combining multiple local SGD steps with periodic model averaging, FedAvg significantly reduces communication costs across clients [McM+17].

Algorithm 1 Federated Averaging (FedAvg) [McM+17]

```

1: Input: Initial global model  $w^0$ , number of rounds  $T$ , fraction of clients  $C$ , local epochs  $E$ , batch
   size  $B$ , learning rate  $\eta$ 
2: for each round  $t = 0, 1, \dots, T - 1$  do
3:   Server selects a subset of clients  $S_t$  with fraction  $C$ 
4:   Server sends current global model  $w^t$  to all  $i \in S_t$ 
5:   for each client  $i \in S_t$  in parallel do
6:      $w_i^{t,0} \leftarrow w^t$ 
7:     for local epoch  $e = 1, \dots, E$  do
8:       for batch  $b \in D_i$  do
9:          $w_i^t \leftarrow w_i^t - \eta \nabla l(w_i^t; b)$ 
10:      end for
11:    end for
12:    Client  $i$  returns updated model  $w_i^{t+1}$ 
13:  end for
14:  Server aggregates:
      
$$w^{t+1} \leftarrow \sum_{i \in S_t} \frac{|D_i|}{\sum_{j \in S_t} |D_j|} w_i^{t+1}$$

15: end for
16: Return: Final global model  $w^T$ 

```

In FedAvg, local updates are computed through multiple steps of SGD on client-specific objectives, causing model updates to be evaluated at different points in parameter space. As a result, the aggregated update no longer corresponds to the gradient of the global objective evaluated at a common model, introducing bias. This discrepancy is mitigated when the number of local steps is small or when client data distributions are similar, in which case FedAvg approximates FedSGD.

2.2 Personalized Federated learning

Personalized federated learning is a branch of federated learning that addresses the challenge of data heterogeneity across clients by enabling the training of models tailored to individual user tasks. More specifically, PFL is relevant when heterogeneity is not only an optimization difficulty for learning a single global model, but also reflects differences in the prediction tasks or local data distributions required by individual clients. As highlighted in [FMO20], the growing interest in PFL is largely driven by its applicability in domains such as personalized healthcare and content recommendation. The authors further note that recent approaches have leveraged model-agnostic meta-learning (MAML) techniques, which facilitate rapid adaptation to new clients using only a limited number of data samples, thereby improving the degree of personalization. In contrast, traditional federated learning seeks to learn a single global model by aggregating information across all client datasets, with the aim of minimizing an average global objective and achieving strong generalization over the collective client data distribution [FMO20]. However, in the presence of non-IID data, individual clients may exhibit significantly different

underlying data distributions, causing their local updates to become biased toward local objectives and to vary substantially across clients. This can slow or destabilize convergence when the goal remains a single global model [Li+20b; Kar+20]. Consequently, a model obtained by minimizing the average global loss may fail to provide adequate performance for specific users, particularly in scenarios where the objective shifts from learning one globally generalizable predictor to learning models that are well suited to each client’s specific task or data distribution [FMO20].

Per-FedAvg Algorithm

The Per-FedAvg algorithm is a PFL method that builds upon the model-agnostic meta-learning (MAML) framework introduced by [FAL17]. MAML is a meta-learning approach designed to be model-agnostic, meaning it can be applied to any model trained using gradient-based optimization. While its primary focus is on neural networks, the method is sufficiently flexible to accommodate a variety of model architectures and problem settings, including classification, regression, and reinforcement learning, with minimal modification. The central objective of meta-learning in this context is to train a model that can rapidly adapt to new tasks using only a small number of task-specific data samples [FAL17]. The key idea underlying this approach is to learn an initialization of model parameters such that, after one or a few gradient update steps computed on limited data from a new task, the adapted model achieves strong performance on that task. [FMO20] implement this MAML concept and apply it at the federated level and solve the following objective:

$$\min_{w \in R^d} F(w) := \frac{1}{n} \sum_{i=1}^n f_i(w - \alpha \nabla f_i(w)) \quad (2.3)$$

where $\alpha > 0$ is the adaptation step size, which controls how strongly the model adapts to a client’s local data. Larger values of α lead to more aggressive weight updates, resulting in stronger personalization toward the local dataset. In contrast, smaller values produce only minor deviations from the global model parameters, thereby limiting personalization since the updates become nearly negligible. The authors propose the Per-FedAvg algorithm to solve the problem (2.3). This formulation shifts the optimization objective from minimizing the global loss to minimizing the expected performance after local adaptation. Consequently, the learned model parameters serve as an initialization that is explicitly optimized for fast personalization on each client. The method extends FedAvg by optimizing the meta-objective in (2.3) rather than the standard global loss. The clients compute the gradient of the local functions $F_i(w)$ which is defined as

$$F_i(w) := f_i(w - \alpha \nabla f_i(w)) \quad (2.4)$$

$$\nabla F_i(w) = (I - \alpha \nabla^2 f_i(w)) \nabla f_i(w - \alpha \nabla f_i(w)) \quad (2.5)$$

However, computing the gradient of f_i at each training round is computationally heavy. Hence the algorithm uses a batch of the local data D^i to obtain the estimate $\tilde{\nabla} f_i(w, D^i)$. Therefore

$$\tilde{\nabla} F_i(w_{k+1,t-1}^i) := (I - \alpha \tilde{\nabla}^2 f_i(w_{k+1,t-1}^i, D_t^i)) \tilde{\nabla} f_i(w_{k+1,t-1}^i - \alpha \tilde{\nabla} f_i(w_{k+1,t-1}^i, D_t^i), D_t^i) \quad (2.6)$$

where D^i, D^i, D^i are independent batches. Similar to FedAvg, at each round k the server sends the current rounds global model weights w_k to a subset of the total clients A_k chosen at random with a size of rn where n is the total number of clients. Each user then locally computes τ steps of gradient descent (GD) with F_i as the loss function. These local updates form the sequence $\{w_{k+1,t}^i\}_{t=0}^\tau$ where $w_{k+1,0}^i = w_k$ for $\tau \geq t \geq 1$

$$w_{k+1,t}^i = w_{k+1,t-1}^i - \beta \tilde{\nabla} F_i(w_{k+1,t-1}^i) \quad (2.7)$$

where β is the learning rate. Once the final local round is complete and $w_{k+1,\tau}^i$ is computed, then each client sends the final update back to the server to be aggregated and compute the new global models weights.

$$w_{k+1} = \frac{1}{rn} \sum_{i \in A_k} w_{k+1,\tau}^i$$

Algorithm 2 The proposed Personalized FedAvg (Per-FedAvg) Algorithm

```

1: Input: Initial iterate  $w_0$ , fraction of active users  $r$ 
2: for  $k = 0$  to  $K - 1$  do
3:   Server chooses a subset of users  $\mathcal{A}_k$  uniformly at random with size  $rn$ 
4:   Server sends  $w_k$  to all users in  $\mathcal{A}_k$ 
5:   for all  $i \in \mathcal{A}_k$  do
6:     Set  $w_{k+1,0}^i = w_k$ 
7:     for  $t = 1$  to  $\tau$  do
8:        $w_{k+1,t}^i = w_{k+1,t-1}^i - \beta \tilde{\nabla} F_i(w_{k+1,t-1}^i)$ 
9:     end for
10:    Agent  $i$  sends  $w_{k+1,\tau}^i$  back to server
11:   end for
12:   Server updates:

```

$$w_{k+1} = \frac{1}{rn} \sum_{i \in \mathcal{A}_k} w_{k+1,\tau}^i$$

```

13: end for

```

Algorithm 2 corresponds to the *full* MAML formulation, where the gradient of the meta-objective F_i is computed as shown in (2.6). This requires evaluating the Hessian $\tilde{\nabla}^2 f_i$. In practice, this operation is computationally expensive. When applied over a batch D_t^i of size d , it requires estimating second-order information for each sample, effectively leading to d Hessian computations per local update step t . This significantly increases both computational and memory costs. Such requirements are particularly problematic in federated learning, where client devices are often resource-constrained in terms of processing power and energy. To address this limitation, alternative approximations of MAML are commonly employed. First-order MAML (FO-MAML) eliminates the Hessian term entirely, relying only on first-order gradients, while Hessian-free MAML (HF-MAML) approximates the Hessian-vector product using finite differences rather than explicitly computing the Hessian matrix. These approaches reduce the computational burden while retaining much of the adaptation capability of the full MAML formulation, making them more suitable for federated settings such as Per-FedAvg.

First Order Model Agnostic Meta Learning

To reduce the computational burden of the full MAML formulation, first-order MAML (FO-MAML) introduces a simplification by omitting the second-order derivative term in (2.6). Specifically, the Hessian term $\nabla^2 f_i(w)$ is ignored, which corresponds to approximating

$$(I - \alpha \nabla^2 f_i(w)) \approx I. \quad (2.8)$$

Under this approximation, the gradient of the meta-objective simplifies to

$$\nabla F_i(w) \approx \nabla f_i(w - \alpha \nabla f_i(w)). \quad (2.9)$$

In the stochastic setting used in Per-FedAvg, this yields the estimator

$$\tilde{\nabla} F_i(w) \approx \tilde{\nabla} f_i(w - \alpha \tilde{\nabla} f_i(w, D_t^i), D_t^i), \quad (2.10)$$

which only requires first-order gradient evaluations. This approximation significantly reduces computational and memory complexity, as it avoids explicit Hessian computation while retaining the bilevel optimization structure of MAML. As a result, FO-MAML is particularly well-suited for federated learning environments with limited client resources, albeit at the cost of introducing approximation error in the meta-gradient [FAL17].

Hessian Free Model Agnostic Meta Learning

Hessian-free MAML (HF-MAML) aims to reduce the computational complexity of the full MAML formulation while still incorporating second-order information. A key limitation of standard MAML is that the computation of the meta-gradient requires evaluating second-order derivatives, which can be difficult and computationally expensive in practice [Son+20]. To address this, HF-MAML avoids explicit computation of the Hessian by approximating Hessian-vector products using finite differences of gradients. In particular, for a vector v , the Hessian-vector product can be approximated as

$$\nabla^2 f_i(w)v \approx \frac{\nabla f_i(w + \delta v) - \nabla f_i(w - \delta v)}{2\delta}, \quad (2.11)$$

where $\delta > 0$ is a small scalar. Substituting this approximation into the meta-gradient expression yields

$$\nabla F_i(w) \approx (I - \alpha \tilde{H}_i(w)) \nabla f_i(w - \alpha \nabla f_i(w)), \quad (2.12)$$

where $\tilde{H}_i(w)$ denotes the finite-difference approximation of the Hessian. This approach preserves curvature information without explicitly forming the Hessian matrix, thereby significantly reducing computational and memory requirements. As a result, HF-MAML provides a practical compromise between the efficiency of first-order methods and the accuracy of full second-order MAML, making it well-suited for resource-constrained federated learning settings such as Per-FedAvg.

FedPer Algorithm

FedPer is a personalized federated learning algorithm proposed by [Ari+19] to address the effects of statistical heterogeneity in federated training. The central idea is to decompose each client model into two components: a set of *base layers*, which are shared across all clients and trained collaboratively, and a set of *personalization layers*, which remain local to each client and are trained only on that client's private data [Ari+19]. In this way, the shared base layers capture common feature representations across the population, while the personalization layers adapt the model to user-specific preferences or local data characteristics.

Formally, let W_B denote the parameters of the shared base layers and let W_{P_j} denote the parameters of the personalization layers of client j , where $j \in \{1, 2, \dots, N\}$ and N is the number of participating clients. The prediction model on client j can then be written as

$$\hat{y} = f(x; W_B, W_{P_j}),$$

where x is the input data and $\hat{y}$ is the corresponding model output. Since each client has its own personalization layers, the overall learning objective is no longer the minimization of a single global loss over one common model. Instead, the goal is to minimize the average personalized population risk across all clients:

$$L_{PR}(W_B, W_{P_1}, \dots, W_{P_N}) = \frac{1}{N} \sum_{j=1}^N E_{(x,y) \sim P_j} [l(y, f(x; W_B, W_{P_j}))], \quad (2.13)$$

where P_j denotes the data distribution of client j and $l(\cdot, \cdot)$ is the per-sample loss function.

Because the true client distributions P_j are unknown in practice, FedPer optimizes the empirical risk on each client [Ari+19]. If client j possesses n_j local training examples $\{(x_{j,i}, y_{j,i})\}_{i=1}^{n_j}$, then its local empirical risk is defined as

$$L_j^{ER}(W_B, W_{P_j}) = \frac{1}{n_j} \sum_{i=1}^{n_j} l(y_{j,i}, f(x_{j,i}, W_B, W_{P_j})). \quad (2.14)$$

Thus, FedPer can be interpreted as jointly optimizing a shared representation through W_B while simultaneously learning client-specific model components through W_{P_j} .

The training procedure follows a hybrid local-global strategy. At the beginning of training, the server initializes the shared base-layer parameters $W_B^{(0)}$ and each client initializes its personalization-layer parameters $W_{P_j}^{(0)}$. During each global round k , the server broadcasts the current shared base parameters $W_B^{(k-1)}$ to all active clients. Client j then performs local stochastic gradient descent on its own empirical risk in (2.14), updating both the received base layers and its private personalization layers:

$$\left(W_{B,j}^{(k)}, W_{P_j}^{(k)}\right) \leftarrow \text{SGD}_j\left(W_B^{(k-1)}, W_{P_j}^{(k-1)}, \eta_j^{(k)}\right),$$

where $\eta_j^{(k)}$ denotes the local learning rate used by client j at round k . After local training, the client sends only its updated base-layer parameters $W_{B,j}^{(k)}$ back to the server, while the personalization-layer parameters $W_{P_j}^{(k)}$ remain stored locally and are never shared [Ari+19].

Once the server receives the updated base layers from all active clients, it aggregates them using a weighted average proportional to the local dataset sizes, following the standard federated averaging paradigm [McM+17]:

$$W_B^{(k)} = \sum_{j=1}^N \gamma_j W_{B,j}^{(k)}, \quad \gamma_j = \frac{n_j}{\sum_{m=1}^N n_m}. \quad (2.15)$$

The aggregated parameters $W_B^{(k)}$ then become the shared base model for the next communication round. In contrast, the personalization parameters are never averaged, since they are intended to preserve client-specific information [Ari+19].

This distinction from FedAvg is fundamental. In FedAvg, the entire model is shared and averaged across clients, which enforces a single common model for all users [McM+17]. In FedPer, only the base layers participate in federated aggregation, whereas the personalization layers are optimized solely with local data. As a result, FedPer learns overlapping but non-identical models across clients. This makes it particularly suitable for tasks in which the same input may correspond to different labels or preferences across users, since such variation cannot be adequately captured by a single global model [Ari+19].

The motivation for this design is that earlier layers of deep networks tend to learn general-purpose feature representations, while later layers capture task- or user-specific decision boundaries. By keeping the personalization layers local, FedPer allows different clients to adapt the shared representation to their own data distribution without forcing complete model agreement across the federation [Ari+19].

The complete FedPer training procedure is summarized in Algorithm 3. For clarity, the algorithm below presents the server-side aggregation and the client-side local update jointly.

Algorithm 3 Federated Personalization with Base and Personalization Layers (FedPer) [Ari+19]

-
- 1: **Input:** Initial shared base parameters $W_B^{(0)}$, local personalization parameters $\{W_{P_j}^{(0)}\}_{j=1}^N$, local learning rates $\{\eta_j^{(k)}\}$, number of global rounds K
 - 2: Each client j sends its local dataset size n_j to the server
 - 3: Server computes aggregation weights $\gamma_j = \frac{n_j}{\sum_{m=1}^N n_m}$
 - 4: **for** $k = 1, 2, \dots, K$ **do**
 - 5: Server broadcasts $W_B^{(k-1)}$ to all active clients
 - 6: **for all** clients j **in parallel do**
 - 7: Client j initializes local model with $(W_B^{(k-1)}, W_{P_j}^{(k-1)})$
 - 8: Client j performs local SGD on $L_j^{ER}(W_B, W_{P_j})$
 - 9: Obtain updated parameters:

$$(W_{B,j}^{(k)}, W_{P_j}^{(k)}) \leftarrow \text{SGD}_j(W_B^{(k-1)}, W_{P_j}^{(k-1)}, \eta_j^{(k)})$$

- 10: Client j sends only $W_{B,j}^{(k)}$ to the server
- 11: **end for**
- 12: Server aggregates shared base layers:

$$W_B^{(k)} \leftarrow \sum_{j=1}^N \gamma_j W_{B,j}^{(k)}$$

- 13: **end for**
 - 14: **Return:** Shared base model $W_B^{(K)}$ and local personalization models $\{W_{P_j}^{(K)}\}_{j=1}^N$
-

By separating globally shared and locally personalized parameters, FedPer offers a simple but effective mechanism for handling client-level statistical heterogeneity in deep federated learning. It preserves the collaborative benefit of federated training through the shared base layers, while allowing each client to retain a personalized decision function through its private layers. This makes the method especially relevant in settings where user preferences or local data semantics differ substantially across clients [Ari+19].

2.3 Gradient Inversion Attacks

Given FL's privacy-by-design paradigm, where raw training data remains local to each client, it is assumed that sharing gradients or model updates will not expose sensitive information. However, [ZLH19] demonstrated that gradients themselves can leak significant information about the underlying training data. Their work introduced the Deep Leakage from Gradients (DLG) attack, which is widely regarded as the first practical semi-honest inference attack in federated learning.

DLG belongs to the class of Gradient Inversion Attacks (GIA), where an adversary attempts to reconstruct private training data from shared gradients or model updates. These attacks operate under a semi-honest (honest-but-curious) threat model, meaning the attacker follows the protocol correctly but passively observes available information to infer sensitive data. No protocol deviation or active manipulation is required.

Deep Leakage from Gradients

The core idea of DLG, proposed by [ZLH19], is to treat gradient inversion as an optimization problem. Suppose an attacker observes the gradients $\nabla_W l(F(x; W), y)$ computed on a private data point (x, y) . The attacker initializes a dummy input x and dummy label y randomly, and seeks to recover the original data by minimizing the discrepancy between the gradients generated by (x, y) and the observed gradients. Formally, the reconstruction objective is given by

$$\min_{x, y} \|\nabla_W l(F(x; W), y) - \nabla_W l(F(x; W), y)\|_F^2. \quad (2.16)$$

This formulation relies on the fact that gradients encode rich information about both the input features and the label. In particular, for sufficiently expressive models, the mapping from (x, y) to $\nabla_W l(F(x; W), y)$ can be highly informative, making inversion feasible. The optimization is carried out iteratively using gradient-based methods. At each step, the attacker computes the gradients induced by the current dummy pair (x, y) , evaluates the gradient matching loss, and updates both variables using gradient descent:

$$x \leftarrow x - \eta \nabla_x L_G, \quad y \leftarrow y - \eta \nabla_y L_G, \quad (2.17)$$

where

$$L_G = \|\nabla_W l(F(x; W), y) - \nabla_W l(F(x; W), y)\|_F^2. \quad (2.18)$$

Unlike standard training, where labels are fixed and inputs are given, DLG jointly optimizes both the input and the label. To enable gradient-based optimization over labels, the discrete label y is typically relaxed into a continuous representation (e.g., a probability vector), allowing differentiation through the loss function.

An important property exploited by DLG is that gradients are highly sensitive to both the input and label. Even small changes in either can lead to noticeable differences in the gradient, which allows the optimization process to gradually align the dummy gradients with the true gradients. Empirically, [zhu2019dlg] show that this process can recover high-fidelity approximations of the original training data, especially when the batch size is small or consists of a single data point.

However, the joint optimization over (x, y) introduces significant challenges. The optimization landscape is highly non-convex, and the discrete nature of labels makes convergence unstable. As noted in subsequent work, DLG often struggles to correctly recover the ground-truth label and may converge to incorrect or ambiguous reconstructions. These limitations motivate the development of improved methods such as iDLG, which decouple label extraction from input reconstruction.

Despite these challenges, DLG provides a fundamental result: gradients alone can leak both input data and supervisory signals. This demonstrates that sharing gradients in federated learning systems can pose a serious privacy risk, even under the honest-but-curious threat model, and establishes gradient inversion as a critical attack vector in distributed learning settings.

Algorithm 4 Deep Leakage from Gradients (DLG) [zhu2019dlg]

```

1: Input: Differentiable model  $F(x; W)$ , shared gradients  $\nabla_W$ , maximum iterations  $N$ , learning rate  $\eta$ 
2: Output: Reconstructed dummy input  $x$  and label  $y$ 
3: Initialize dummy input  $x \sim \mathcal{N}(0, 1)$  and dummy label  $y \sim \mathcal{N}(0, 1)$ 
4: for  $t = 1, 2, \dots, N$  do
5:   Compute dummy gradients:  $\nabla_{W'} \leftarrow \frac{\partial l(F(x'; W), y')}{\partial W}$ 
6:   Compute gradient matching loss:  $L_G = \|\nabla_{W'} - \nabla_W\|_F^2$ 
7:   Update dummy variables:  $x' \leftarrow x' - \eta \nabla_{x'} L_G, \quad y' \leftarrow y' - \eta \nabla_{y'} L_G$ 
8: end for
9: Return:  $x, y$ 

```

Inverting Gradients Attack

While early gradient inversion attacks such as DLG demonstrate that gradients leak information about training data, they are often limited in terms of reconstruction quality and optimization stability. More recent work shows that these limitations are not inherent to the problem. In particular, the Inverting Gradients (IG) attack demonstrates that accurate reconstruction is possible even for deep, trained neural networks under realistic conditions [Gei+20].

As in DLG, the attack is formulated under a semi-honest (honest-but-curious) threat model, where an adversary observes gradients shared during training and attempts to reconstruct the underlying private data. Let $\nabla_\theta \mathcal{L}_\theta(x^*, y)$ denote the observed gradient corresponding to an unknown data sample (x^*, y) . The attacker initializes a dummy input x and optimizes it such that the generated gradients match the observed gradients.

A key difference from DLG lies in the formulation of the reconstruction objective. Instead of using a simple Euclidean distance between gradients, IG introduces a magnitude-invariant similarity measure based on cosine similarity. The reconstruction objective is defined as:

$$\min_x 1 - \frac{\langle \nabla_\theta \mathcal{L}_\theta(x, y), \nabla_\theta \mathcal{L}_\theta(x^*, y) \rangle}{\|\nabla_\theta \mathcal{L}_\theta(x, y)\| \|\nabla_\theta \mathcal{L}_\theta(x^*, y)\|}. \quad (2.19)$$

This formulation removes the dependence on the magnitude of the gradients and instead focuses on their direction. This is important in practice, as gradient magnitudes can vary significantly across layers and training iterations, which can destabilize optimization when using an ℓ_2 objective.

In addition to the gradient matching term, the attack incorporates a regularization term to enforce natural image structure in the reconstruction. Specifically, total variation (TV) regularization is added to penalize high-frequency noise:

$$L_{\text{TV}}(x) = \sum_{i,j} ((x_{i,j+1} - x_{i,j})^2 + (x_{i+1,j} - x_{i,j})^2). \quad (2.20)$$

The final optimization objective can therefore be written as:

$$\min_x L_{\text{grad}}(x) + \lambda L_{\text{TV}}(x), \quad (2.21)$$

where L_{grad} denotes the cosine similarity loss and λ controls the strength of the regularization term.

The optimization is performed iteratively using gradient-based updates on the dummy input. Compared to DLG, this formulation results in significantly more stable convergence and produces higher-quality

reconstructions, particularly for deep architectures [Gei+20].

Algorithmically, the attack follows a similar structure to DLG, but with the modified loss function:

Algorithm 5 IG Attack [Gei+20]

- 1: **Input:** Model $\mathcal{L}_\theta$, observed gradients ∇_θ , iterations T , learning rate η , regularization weight λ
 - 2: **Output:** Reconstructed input x
 - 3: Initialize dummy input $x \sim \mathcal{N}(0, 1)$
 - 4: **for** $t = 1, 2, \dots, T$ **do**
 - 5: Compute dummy gradients:

$$\nabla'_\theta \leftarrow \nabla_\theta \mathcal{L}_\theta(x, y)$$
 - 6: Compute cosine similarity loss:

$$L_{\text{grad}} = 1 - \frac{\langle \nabla'_\theta, \nabla_\theta \rangle}{\|\nabla'_\theta\| \|\nabla_\theta\|}$$
 - 7: Compute total variation regularization:

$$L_{\text{TV}} = \sum_{i,j} ((x_{i,j+1} - x_{i,j})^2 + (x_{i+1,j} - x_{i,j})^2)$$
 - 8: Combine losses:

$$L = L_{\text{grad}} + \lambda L_{\text{TV}}$$
 - 9: Update dummy input:

$$x \leftarrow x - \eta \nabla_x L$$
 - 10: **end for**
 - 11: **Return:** x
-

Beyond the numerical optimization perspective, the attack also provides theoretical insight into why gradient inversion is possible. In particular, it is shown that for networks containing fully connected layers, the input to such layers can be reconstructed analytically from gradient information under mild assumptions [Gei+20]. This implies that certain architectural components inherently leak information, independent of the rest of the network.

Another important observation is that increasing model depth does not necessarily improve privacy. Deep neural networks remain vulnerable to inversion, and experimental results show that even large architectures trained on high-resolution datasets can be reconstructed with high fidelity [Gei+20].

Overall, IG demonstrates that gradients should not be considered inherently privacy-preserving, and that additional mechanisms are required to mitigate information leakage in federated learning systems.

2.4 Convolutional Neural Networks

Convolutional neural networks are specifically designed to process data with a grid-like structure, such as images. The key idea is to replace fully connected operations with structured, local computations that exploit spatial correlations. A CNN is typically composed of three main types of layers: convolutional layers, pooling (subsampling) layers, and fully connected layers.

Convolution Operation

The core operation in a CNN is the convolution. Given an input image $x \in R^{H \times W \times C}$ and a filter (kernel) $K \in R^{k \times k \times C}$, the convolution operation produces a feature map h defined as:

$$h_{i,j} = \sum_{c=1}^C \sum_{u=1}^k \sum_{v=1}^k K_{u,v,c} \cdot x_{i+u-1,j+v-1,c} + b, \quad (2.22)$$

where b is a bias term. This operation is applied across all spatial locations (i, j) of the input, producing a 2D feature map.

Each filter learns to detect a specific pattern (e.g., edges, corners), and multiple filters produce multiple feature maps. This process can be interpreted as a parameter-efficient way to learn local feature extractors.

Activation Functions

After convolution, a nonlinear activation function $\sigma(\cdot)$ is applied element-wise:

$$h'_{i,j} = \sigma(h_{i,j}). \quad (2.23)$$

Weight Sharing and Local Connectivity

Unlike fully connected networks, CNNs enforce:

- Local connectivity: each neuron depends only on a small region (receptive field) of the input.
- Weight sharing: the same filter K is applied across all spatial locations.

This reduces the number of parameters and introduces translation invariance, as the same feature can be detected regardless of its position in the image [Lec+98].

Pooling (Subsampling)

Pooling layers reduce the spatial resolution of feature maps. In LeNet, average pooling is used. For a feature map h , pooling computes:

$$p_{i,j} = \frac{1}{s^2} \sum_{u=1}^s \sum_{v=1}^s h_{(i-1)s+u, (j-1)s+v}, \quad (2.24)$$

where s is the pooling window size. Pooling improves robustness to small translations and reduces computational cost.

Fully Connected Layers

After several convolution and pooling stages, the feature maps are flattened into a vector and passed through fully connected layers:

$$z = Wx + b, \quad (2.25)$$

where W is a weight matrix and b is a bias vector. These layers perform high-level reasoning and map extracted features to class scores.

Output Layer

For classification, the final layer uses the softmax function to produce class probabilities:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \quad (2.26)$$

The model is trained using a loss function such as cross-entropy:

$$\mathcal{L}(x, y) = - \sum_i y_i \log(\hat{y}_i). \quad (2.27)$$

Gradient-Based Learning

All parameters θ of the network are optimized using gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}, \quad (2.28)$$

where η is the learning rate. Gradients are computed efficiently using backpropagation [Lec+98].

2.5 Performance Metrics

The objective of the evaluation is to quantify the accuracy with which an attacker can reconstruct the original input data. To assess reconstruction quality, three complementary metrics are employed: Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Structural Similarity Index Measure (SSIM). Each metric captures a different aspect of reconstruction fidelity—pixel-wise error, signal quality, and perceptual similarity, respectively. Together, they provide a comprehensive and balanced evaluation of the effectiveness of the gradient inversion attacks.

Mean Squared Error

Mean Squared Error (MSE) is a widely used full-reference metric that measures the average squared difference between the original image x and the reconstructed image $\hat{x}$. It is defined as:

$$\text{MSE}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2, \quad (2.29)$$

where n denotes the total number of pixels. MSE provides a direct measure of pixel-wise reconstruction error, where lower values indicate higher similarity between the reconstructed and original images. However, as noted in [HZ10], MSE does not account for perceptual quality and may fail to reflect how humans visually interpret image similarity.

Peak Signal to Noise Ratio

Peak Signal-to-Noise Ratio (PSNR) is derived from MSE and expresses reconstruction quality as the ratio between the maximum possible signal value and the distortion introduced by noise. It is typically measured in decibels (dB) and defined as:

$$\text{PSNR}(x, \hat{x}) = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}(x, \hat{x})} \right), \quad (2.30)$$

where MAX is the maximum possible pixel value (e.g., 255 for 8-bit images). Higher PSNR values

indicate better reconstruction quality. While PSNR improves interpretability over MSE by using a logarithmic scale, it remains fundamentally based on pixel-wise differences and therefore shares similar limitations in capturing perceptual similarity [HZ10].

Structural Similarity Index Measure

Structural Similarity Index Measure (SSIM) is a perceptual metric designed to better align with human visual perception by comparing structural information between images. Unlike MSE and PSNR, SSIM decomposes image similarity into three components: luminance, contrast, and structure. It is defined as:

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}, \quad (2.31)$$

where μ_x and $\mu_{\hat{x}}$ are the mean intensities, σ_x^2 and $\sigma_{\hat{x}}^2$ are the variances, and $\sigma_{x\hat{x}}$ is the covariance between the images. Constants C_1 and C_2 are included to stabilize the division.

SSIM values typically lie in the range $[-1, 1]$, with values closer to 1 indicating higher similarity. As highlighted by [HZ10], SSIM better captures perceptual image quality by focusing on structural information rather than absolute pixel differences, making it particularly suitable for evaluating reconstruction fidelity in image-based attacks.

3.1 Research Objective and Experimental Overview

The objective of this thesis is to investigate the feasibility of gradient inversion attacks in the context of personalized federated learning. While federated learning is commonly regarded as a privacy-preserving paradigm, prior work has demonstrated that gradients and model updates can encode substantial information about the underlying training data, enabling potential reconstruction of private inputs [ZLH19]. The introduction of personalization mechanisms further modifies the structure of these updates, raising the question of whether such approaches mitigate or instead exacerbate information leakage. The central research question of this work is formulated as follows:

How do personalization strategies in federated learning influence the success and reconstruction quality of gradient inversion attacks compared to standard non-personalized federated learning algorithms?

To address this question, a controlled experimental framework is constructed in which a client performs local training and transmits model updates to a central server. An adversary, operating under an honest-but-curious threat model, observes these updates and attempts to reconstruct the client's private data. The quality of reconstruction is evaluated across different algorithmic configurations.

The experimental design consists of a systematic comparison between three classes of algorithms: standard federated averaging (FedAvg), layer-based personalization (FedPer), and meta-learning-based personalization (Per-FedAvg). Each of these methods introduces distinct structural properties in the update rule, thereby altering the information exposed through shared updates. By isolating these structural differences, the study aims to attribute variations in reconstruction performance of PFL's directly to algorithmic design choices rather than external factors and compare them directly to the base FL case. To ensure a controlled analysis, the experimental setup is deliberately constrained to a minimal setting consisting of a single local optimization step and a batch size of one. This configuration removes the effects of gradient aggregation, multi-step optimization dynamics, and batch averaging, all of which are known to obscure or dilute information leakage in practical deployments [Dim+22]. While such a setting does not reflect real-world training conditions, it enables the isolation of the intrinsic leakage properties induced by the algorithmic structure itself. In this sense, the experimental design serves as an analytical baseline.

The empirical evaluation focuses on an image classification task using a convolutional neural network (CNN). The model is trained under the different federated learning algorithms, and gradient inversion attacks are applied to the resulting updates to assess reconstruction performance. This setup provides a consistent and controlled environment for comparing the susceptibility of each algorithm to information leakage.

The complete implementation of the experimental framework is available at:

<https://github.com/rt-k3498/PFL-attack-simulation.git>

3.2 Algorithm Design

The experiment compares three federated learning algorithm classes under the same simulation structure: FedAvg, FedPer, and Per-FedAvg. For each algorithm, clients receive the current server model, perform local training on one private sample, and return the resulting update to the server. The local loss is categorical cross-entropy. All algorithms use one local training step per communication round. Aggregation is performed by an unweighted elementwise average over the participating client updates.

FedAvg

FedAvg is implemented as the non-personalized federated learning baseline. At communication round r , the server sends the current global model parameters w^r to each client. Client i performs one local gradient descent step (with alpha/step-size of 0.1) on its selected private sample. The full updated model weights are returned to the server. After all client updates are collected, the server computes the next global model by averaging the submitted weights. No weighting by local dataset size is applied, as the dataset size used for local training is equal amongst the client in all communication rounds, in the implementation.

FedPer

FedPer is implemented as the layer-based personalization method. The model parameters are separated into shared base parameters and private personalization parameters:

$$w_i = (w_B, w_{P_i}), \quad (3.1)$$

where w_B denotes the shared base-layer parameters and w_{P_i} denotes the personalization-layer parameters stored locally by client i . The model used in the experiment has six trainable layers. FedPer is evaluated with

$$K_p \in \{1, 2, 3, 4, 5\}, \quad (3.2)$$

where K_p is the number of final trainable layers kept private. The remaining $6 - K_p$ trainable layers are shared with the server. Since each trainable layer contains a kernel and bias tensor, K_p private layers correspond to $2K_p$ private weight arrays.

Variant	Private trainable layers	Shared trainable layers
FedPer($K_p = 1$)	1	5
FedPer($K_p = 2$)	2	4
FedPer($K_p = 3$)	3	3
FedPer($K_p = 4$)	4	2
FedPer($K_p = 5$)	5	1

Table 3.1: FedPer layer splits used in the experiment.

At round r , the server sends the entire model parameters to each client. Client i combines only the base parameters w_b with its local personalization parameters $w_{P_i}^r$ (this was a programmatic implementation decision that effectively behaves similar to just sending the w_b weights) and performs one local update (with alpha/step-size of 0.1) on the full model. Only $w_{B,i}^{r,+}$ is returned to the server. The personalization parameters $w_{P_i}^{r+1}$ remain local to the client and are reused in later communication rounds. The server aggregates only the shared base-layer parameters. Thus, FedPer modifies the server-visible update by removing the final K_p trainable layers from communication and aggregation.

Per-FedAvg

Per-FedAvg is implemented as the meta-learning-based personalization method. The experiment evaluates three update variants: first-order Per-FedAvg, Hessian-free Per-FedAvg, and exact Hessian-vector-product Per-FedAvg. All variants use one inner adaptation step (with alpha of 0.1) and one outer update step (with beta of 0.1). In the implementation, the same private sample (x_i^r, y_i^r) is used for both the adaptation step and the meta-update.

3.3 Attack Design

The attacks are applied inside the federated training loop after local client training and before server aggregation. Therefore, each attack targets an individual client update. The attacker is modeled as an honest-but-curious server or observer. It follows the communication protocol but attempts to reconstruct the private sample used by the client. The attacker is assumed to know the model architecture, the global model weights before local training, the client-submitted weights after local training, the learning rate used to produce the update, and the number of output classes.

Before each attack execution, the attack initialization is reseeded using the trial seed

$$s_{\text{trial}} = s + 1000 \cdot r_{\text{run}}, \quad (3.3)$$

where $s = 42$ is the global seed and r_{run} is the run index. This keeps the attack initialization fixed across algorithm configurations.

Deep Leakage Gradients

The DLG implementation receives the known model parameters θ^r and the observed update direction q_i^r . The attack optimizes dummy image logits z_x and dummy label logits z_y . These variables are transformed as

$$\hat{x} = \sigma(z_x), \quad \hat{y} = \text{softmax}(z_y), \quad (3.4)$$

so that the reconstructed image remains in the normalized pixel range and the reconstructed label is represented as a probability vector over the CIFAR-10 classes.

For a dummy pair $(\hat{x}, \hat{y})$, the attack computes gradients with respect to the server-visible parameters $\mathcal{V}$:

$$\hat{q}_i^r = \nabla_{\theta_{\mathcal{V}}} \ell(f(\hat{x}; \theta^r), \hat{y}). \quad (3.5)$$

The DLG objective used in the implementation is

$$\mathcal{L}_{\text{DLG}} = \sum_{j \in \mathcal{V}} \|\hat{q}_{i,j}^r - q_{i,j}^r\|_2^2. \quad (3.6)$$

For FedPer, the summation is restricted to the shared-layer gradient tensors. No private personalization-layer tensors are included in the attack objective. The objective is optimized using the TensorFlow Probability implementation of L-BFGS. The output of the attack is the reconstructed image $\hat{x}$ and reconstructed label distribution $\hat{y}$. The reconstructed class is obtained by applying arg max to $\hat{y}$.

IG

The IG implementation uses the same attacker inputs as DLG: the known model parameters θ^r and the observed update direction q_i^r . Dummy image logits and dummy label logits are optimized and transformed as

$$\hat{x} = \sigma(z_x), \quad \hat{y} = \text{softmax}(z_y). \quad (3.7)$$

The dummy gradients are computed over the server-visible parameter set $\mathcal{V}$:

$$\hat{q}_i^r = \nabla_{\theta_{\mathcal{V}}} \ell(f(\hat{x}; \theta^r), \hat{y}). \quad (3.8)$$

The visible dummy gradients and observed update direction are flattened before comparison:

$$\hat{u} = \text{vec}(\hat{q}_i^r), \quad u = \text{vec}(q_i^r). \quad (3.9)$$

The gradient matching term is the cosine distance

$$\mathcal{L}_{\text{cos}} = 1 - \frac{\langle \hat{u}, u \rangle}{\|\hat{u}\|_2 \|u\|_2}. \quad (3.10)$$

The implementation adds total variation regularization on the reconstructed image:

$$\mathcal{L}_{\text{IG}} = \mathcal{L}_{\text{cos}} + \lambda_{\text{TV}} \mathcal{L}_{\text{TV}}(\hat{x}). \quad (3.11)$$

The objective is optimized using Adam with a cosine-decayed learning rate, and signed Adam updates are enabled. As with DLG, only the server-visible gradient tensors are included. Thus, for FedPer, the attack objective excludes the private personalization layers. The attack returns the reconstructed image $\hat{x}$ and reconstructed label distribution $\hat{y}$, with the reconstructed class decoded using $\arg \max$.

3.4 Data

The experiments are conducted on the CIFAR-10 dataset. Each sample is a $32 \times 32 \times 3$ image (pixel values range from 0-255) with integer encoded labels (0-9). There are transformations applied to this dataset before use in the experiment: normalization of the x-values (image pixel normalization to range [0,1]) and one-hot-encoding of the integer encoded labels.

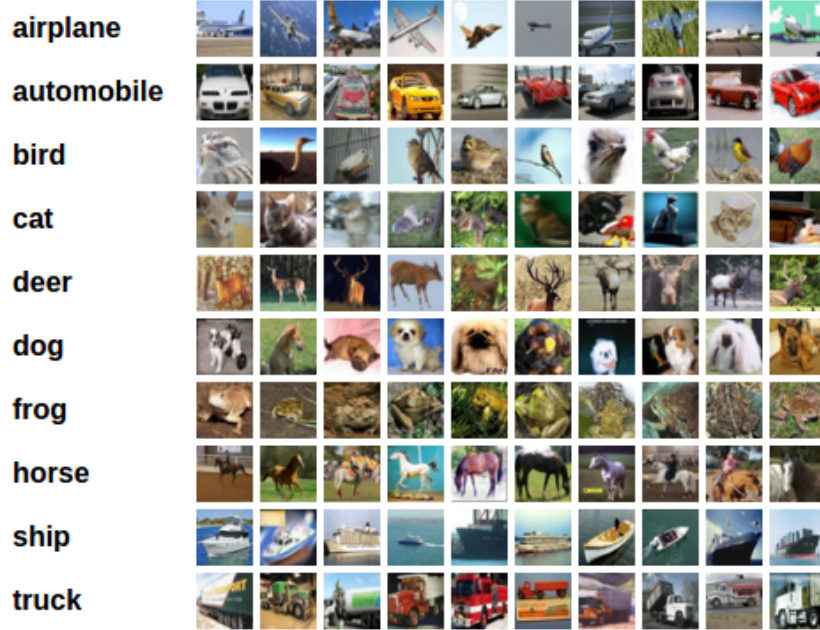


Figure 3.1: CIFAR-10 Dataset.

3.5 Model

The model used in this work is implemented using TensorFlow/Keras as a sequential convolutional neural network. The model operates on input images of shape $(32, 32, 3)$ and produces a probability distribution over 10 classes using a softmax output layer, making it suitable for classification on the CIFAR-10 dataset.

The architecture follows a structured pipeline of convolutional feature extraction followed by dense classification layers. Specifically, the model consists of three convolutional layers interleaved with average pooling layers, followed by a flattening step and three fully connected layers. The convolutional layers use kernel sizes of (5×5) and progressively increase the number of feature maps from 6 to 16 to 120. Pooling layers reduce spatial resolution, resulting in compact feature representations. These are then mapped through dense layers of sizes 256 and 128 before the final classification layer.

All trainable layers are initialized using the Glorot uniform initializer (standard in `keras`) with a fixed seed to ensure reproducibility. Activation functions throughout the network are sigmoid in the hidden layers and softmax at the output.

It should be noted that while the implementation is inspired by LeNet-5 (due to its low computation load and training speed), it represents a modified variant (specific to the requirements of the experimental simulation).

LeNet

LeNet is a class of convolutional neural networks (CNNs) introduced by [Lec+98] for document recognition tasks such as handwritten digit classification. It represents one of the earliest successful applications of gradient-based learning to high-dimensional image data and serves as a foundational architecture for modern deep learning in computer vision.

3.6 Evaluation Metrics

The effectiveness of the reconstruction attacks was evaluated by measuring the fidelity of both the reconstructed input images and the reconstructed labels. Since the attacks considered in this work attempt to recover private client data from communicated model updates, the evaluation must capture two separate aspects of leakage: how closely the reconstructed image matches the original image, and whether the corresponding class label is correctly recovered.

For image reconstruction fidelity, the following three complementary metrics were used: MSE, PSNR, SSIM. These metrics were selected because they evaluate reconstruction quality from different perspectives. MSE is a standard measure of the average pixel-wise squared difference between the original image and the reconstructed image. While MSE provides a direct measure of pixel-level error, it does not provide a standard comparison of errors as it is input scale dependent (not normalized). Therefore, PSNR was also used. SSIM was used to measure the structural similarity between the original and reconstructed images. It is useful for evaluating whether the reconstructed image preserves the visual structure of the original input. Using multiple image reconstruction metrics reduces the risk of drawing conclusions from a single biased measure.

For label reconstruction, a binary correctness metric was implemented. For each attack run, the reconstructed label was compared with the true label. If the reconstructed label matched the original label, the score for that run was assigned a value of 1; otherwise, it was assigned a value of 0:

$$LabelAcc(y, \hat{y}) = \begin{cases} 1, & \text{if } \hat{y} = y, \\ 0, & \text{otherwise.} \end{cases}$$

The final label reconstruction score was computed as the average of this binary value across all runs for a given attack and federated learning algorithm. This produces the empirical label reconstruction accuracy:

$$\text{Label Reconstruction Accuracy} = \frac{1}{R} \sum_{r=1}^R \text{LabelAcc}(y_r, \hat{y}_r),$$

where R is the total number of runs, y_r is the true label in run r , and $\hat{y}_r$ is the reconstructed label.

3.7 Numerical Experiments

The simulation is structured as a controlled privacy experiment that integrates all algorithmic and attack components described in the preceding sections. Privacy is evaluated through the degree of information leakage observed from the attack reconstructions, including both image and label recovery. It should be noted that these measures are not treated as absolute indicators, but rather as comparative metrics relative to a baseline traditional federated learning setting under highly non-IID data conditions.

The central objective of the simulation is to isolate and quantify the effect of federated learning algorithm design on the effectiveness and capabilities of traditional gradient inversion attacks (designed for traditional FL algorithms). To achieve this, the experimental framework is constructed such that all components, except for the learning algorithm and attack method, remain constant across experimental runs. More specifically, all algorithms operate using the same model architecture, identical client data assignments, and identical random seeds. This ensures that any observed variation in reconstruction quality can be attributed directly to differences in algorithmic structure rather than external sources of variation.

Simulation Loop

The outermost loop of the simulation iterates over all algorithm/attack combinations. For each algorithm/attack pair, a fresh model is instantiated using a deterministic initializer and all clients are reinitialized with their fixed local datasets. This ensures that each algorithm/attack trial begins from an identical state.

Within each trial, the simulation proceeds over a fixed number of communication rounds. At the start of each round, the server distributes the current global model to all clients. Each client receives a clone of the global model, ensuring that local updates do not modify the server state during training. The client then trains locally on a single sample retrieved sequentially from its local dataset. The ordering of local samples within each client is deterministic, as samples are shuffled using a NumPy random number generator seeded at initialization. This introduces controlled variability in the order of sample consumption while preserving full reproducibility across runs.

After local training, and before aggregation takes place, the attack is executed. The attacker observes the global model weights before training and the client-submitted weights after training, and recovers an approximation of the client gradient as shown earlier in section 3.3. For FedPer, only the shared base-layer weights are transmitted by the client; accordingly, the attack has access only to the gradient components corresponding to those shared layers. This structural reduction in visible gradient information is a key experimental variable.

Once the attack completes reconstruction, the server aggregates all client-submitted updates to produce the new global model. Aggregation follows an unweighted element-wise average, giving each client

equal contribution to the global model (this is because the client data size used for training in each communication round is constant and equal to a batch size of 1). For FedPer, only the shared base-layer weights participate in this aggregation; private personalization layers are never transmitted and remain stored on each client across rounds.

This sequence of global model distribution, local training, attack execution, and aggregation repeats for all communication rounds.

Algorithm 6 Simulation Loop

Require: Algorithm set $\mathcal{A}$, attack set $\mathcal{K}$, runs M , communication rounds R , clients $\{c_i\}_{i=1}^N$

```

1: for  $m = 1$  to  $M$  do
2:   for each algorithm  $a \in \mathcal{A}$ , attack  $k \in \mathcal{K}$  do
3:     Reinitialize global model  $w^0$  with fixed seed
4:     Reinitialize all clients with fixed local datasets
5:     for  $r = 1$  to  $R$  do
6:       for each client  $c_i$  do
7:          $w_i \leftarrow \text{clone}(w^{r-1})$ 
8:          $(x_i, y_i) \leftarrow c_i.\text{get\_sample}()$ 
9:          $w'_i \leftarrow \text{LocalUpdate}(a, w_i, x_i, y_i)$ 
10:         $g_i \leftarrow (w_i - w'_i)/\alpha$  ▷ Gradient reconstruction
11:         $(\hat{x}_i, \hat{y}_i) \leftarrow \text{Attack}(k, w_i, g_i)$  ▷ Attack before aggregation
12:        Record privacy metrics: MSE, PSNR, SSIM, label comparison
13:      end for
14:       $w^r \leftarrow \text{Aggregate}(\{w'_i\}_{i=1}^N)$ 
15:    end for
16:  end for
17: end for

```

Seeding and Fairness

To ensure reproducibility and a fair comparison across algorithms, the simulation uses a fixed global base seed of 42. This base seed is applied consistently across the dataset construction, client initialization, algorithm setup, attack setup, metric tracking, and model initialization. Randomness is controlled through a shared reseeding routine, which resets the NumPy, TensorFlow, and Python `random` states together. Although the experiment is anchored by one global seed, each run is assigned a deterministic run-specific seed. This is defined as

$$\text{trial_seed}(\text{run}) = 42 + 1000 \cdot \text{run}.$$

For example if the experiments are repeated across three runs, the resulting trial seeds are 42, 1042, and 2042. At the beginning of each run, the corresponding trial seed is applied before the model, clients, algorithm, and attack objects are created or reinitialized. Thus, each run represents a deterministic but distinct experimental trial.

The attack execution is also reseeded immediately before the attack is run. This is handled by the seeded attack wrapper, which resets the random state to the same trial seed directly before calling the attack procedure. As a result, the randomness of DLG and IG depends on the run index rather than on the amount of random state consumed by the preceding training algorithm. This avoids the case where one algorithm could alter the attack initialization simply by consuming a different number of random samples before the attack begins.

This design improves fairness across algorithm/attack comparisons. Within a given run, each algorithm/attack pair starts the attack phase from the same deterministic random conditions. Therefore,

observed differences in reconstruction quality are less likely to be caused by differences in attack initialization and more likely to reflect differences induced by the federated learning algorithm itself. Repeating the full simulation across three deterministic trial seeds reduces the influence of seed specific noise and provides more stable estimates of mean reconstruction performance across algorithm/attack pairs.

Dataset and Client Structure

The experiment uses CIFAR-10, with five clients each assigned a local dataset of ten images drawn from two non-overlapping CIFAR-10 classes, five images per class. The class assignment follows a structured partitioning scheme in which client i receives images from classes i and $i+5$, as summarized in Table 3.2. The constraint

$$N_{\text{clients}} \times \text{labels per client} = 10$$

is enforced by construction, guaranteeing that every CIFAR-10 class is covered exactly once across the client population per run.

Client	Assigned CIFAR-10 classes
0	0, 5
1	1, 6
2	2, 7
3	3, 8
4	4, 9

Table 3.2: Non-IID client data assignment.

Assigning two non-overlapping classes to each client creates a strongly non-IID local data distribution relative to the full CIFAR-10 population. Because class labels do not overlap between clients, each client observes only a restricted subset of the global label space, producing substantial statistical heterogeneity across clients. This design therefore simulates the personalized non-IID setting that arises in high-personalization scenarios, which is the central setting of interest in this thesis.

Each communication round consumes exactly one local image per client via `get_sample()`, which returns samples sequentially from the shuffled local dataset. With ten local samples per client and ten communication rounds, each client exhausts its full local dataset exactly once over the training duration. Consequently, each client transmits exactly one single sample update per round. This design deliberately removes within-batch gradient mixing and multi-step smoothing, constituting the strongest possible setting for gradient inversion. The transmitted update therefore encodes the information of precisely one private training example, serving as an analytical baseline for examining the intrinsic leakage properties of each algorithm’s update structure [Dim+22].

Scale and Output

The full experiment spans three runs, nine algorithm configurations, and two attacks, yielding 2700 attack reconstruction records in total, computed as

$$3 \text{ runs} \times 9 \text{ algorithms} \times 2 \text{ attacks} \times 10 \text{ rounds} \times 5 \text{ clients} = 2700. \quad (3.12)$$

This amounts to 150 reconstructions per algorithm/attack pair, comprising 15 reconstructions per CIFAR-10 class. For each reconstruction, the simulation records the true and reconstructed inputs, the true and reconstructed labels, and the scalar quality metrics MSE, PSNR, and SSIM. These outputs support a systematic comparison of reconstruction quality across all algorithm/attack pairs.

Hyper-parameters

Before conducting the full simulation, a separate hyperparameter tuning experiment was performed for the two gradient inversion attacks. The purpose of this experiment was to calibrate the attacks such that their reconstruction quality was not limited by arbitrary attack settings. Since both DLG and IG are optimization based attacks, their performance depends strongly on parameters such as the optimizer step size, stopping tolerances, line-search budget, L-BFGS correction history, and image regularization strength. If these parameters were chosen randomly, poor reconstruction performance in the main experiment could be caused by weak attack settings rather than by the privacy structure of the learning algorithm. Additionally, their performance on FedAvg will emphasize that the attacks are implemented correctly and that their performance on the PFL algorithms is not due to implementation issues.

The hyperparameter search was therefore treated as a pre-study to the main privacy experiment. The search was performed in a controlled FedAvg setting using one CIFAR-10 image, one client, one local update, and the same LeNet-style model used in the main simulation. The attacker was given the global model before local training and the client model after one local update, from which the gradient signal was reconstructed using the known learning rate. The final selected settings were then fixed and reused in the main simulation across FedAvg, FedPer, and the Per-FedAvg variants. This is important because the main experiment aims to compare algorithmic structure under the same attack configuration, rather than tune a separate "strongest" attack for every algorithm.

For both attacks, the reconstruction budget was fixed to 300 iterations (used to cap the computational requirements and duration of the total experiment). This was also the reconstruction budget later used in the main simulation, ensuring that the hyperparameter tuning was aligned with the final experiment. The hyperparameter search used a coarse-to-fine grid search strategy over three rounds. In each round, candidate settings were evaluated according to reconstruction quality, and the best candidate was selected using the lowest input MSE. The grid was then refined around the best candidate from the previous round to reduce the search scale, producing a local coarse-to-fine refinement procedure. PSNR and SSIM were also recorded, but they were used as supporting metrics rather than as the primary selection criterion.

The DLG initial search grid was

$$\begin{aligned} \text{num_correction_pairs} &\in \{50, 100, 150\}, \\ \text{max_line_search_iterations} &\in \{100, 200, 300\}, \\ \text{tolerance} &\in \{10^{-18}, 10^{-15}, 10^{-12}\}, \\ \text{f_relative_tolerance} &\in \{10^{-18}, 10^{-15}, 10^{-12}\}. \end{aligned}$$

The IG initial search grid was

$$\begin{aligned} \text{init_step_size} &\in \{0.01, 0.1, 1.0\}, \\ \text{final_step_size} &\in \{0.001, 0.01, 0.1\}, \\ \alpha &\in \{10^{-14}, 10^{-13}, 10^{-12}\}. \end{aligned}$$

Here, α is the weight of the total variation regularization term in the attack objective, and should not be confused with the local learning rate used by FedAvg or the inner adaptation step size used by Per-FedAvg.

Since the default image count was one, the reported averages in the hyperparameter search correspond to the reconstruction quality on a single calibration image. This kept the tuning experiment computationally feasible, but it also means that the selected attack settings are highly fitted to that specific calibration example.

Table 3.3: Final hyperparameter values.

Attack	num_correction_pairs	max_line_search_iterations	tolerance	f_relative_tolerance	MSE	PSNR	SSIM
DLG	175	300	1×10^{-15}	1.250×10^{-12}	0.012	19.366	0.795

Attack	init_step_size	final_step_size	alpha	MSE	PSNR	SSIM
IG	0.100	0.088	6.288×10^{-13}	0.008	20.959	0.816

The hyperparameters in Table 3.3 control the numerical optimization procedures used by the two reconstruction attacks. For DLG, `num_correction_pairs` determines how many previous position and gradient correction pairs are stored by the L-BFGS optimizer when approximating the inverse Hessian. A larger value gives the optimizer more curvature information from previous iterations, but also increases memory usage. The parameter `max_line_search_iterations` controls the maximum number of internal line-search steps used to select a suitable step size along each L-BFGS search direction. The `tolerance` parameter defines the convergence threshold for the gradient norm, while `f_relative_tolerance` defines a stopping criterion based on the relative change in the objective value between iterations. Together, these parameters determine how aggressively and precisely the L-BFGS optimizer searches for a dummy input and label whose gradients match the observed client gradients. For IG, `init_step_size` and `final_step_size` define the starting and ending learning rates of the Adam-based reconstruction process. The parameter α weights the image prior or total regularization term relative to the cosine gradient-matching objective. A larger α would enforce the image prior more strongly, while a smaller α makes the reconstruction rely more directly on matching the observed gradient direction.

For DLG, the tuned parameters control the TensorFlow Probability L-BFGS optimizer used to minimize the gradient matching objective. The DLG implementation performed best in the calibration setting when the L-BFGS optimizer was given a relatively large history and a large line-search budget. This is reasonable because the reconstruction problem is jointly non-convex, as the attack optimizes both the dummy image and the dummy label.

For IG, the tuned parameters control the Adam optimization schedule and the image prior. The initial step size was set to 0.100 and the final step size to 0.088, meaning that the cosine decay schedule remained almost flat during the 300 reconstruction iterations. The selected value of α was 6.288×10^{-13} . The very small value means that the attack relied mainly on the cosine gradient-matching objective while only weakly enforcing image smoothness.

This chapter presents the experimental results obtained from the reconstruction attacks evaluated against the federated learning algorithms considered in this thesis. The reported results summarize reconstruction quality, label recovery performance, and the development of these metrics across communication rounds.

The evaluation uses four reconstruction metrics: mean squared error (MSE), peak signal-to-noise ratio (PSNR), structural similarity index measure (SSIM), and label recovery. MSE measures the pixel-level difference between the reconstructed image and the original image. PSNR measures reconstruction quality in terms of signal fidelity, where higher values indicate better reconstructions. SSIM measures structural similarity between the reconstructed and original images, where higher values indicate stronger visual similarity. Label recovery measures whether the attack recovered the correct class label.

Table 4.1: Mean reconstruction performance summary.

Algorithm	Attack	MSE	PSNR	SSIM	Label Recovery
FedAvg	DLG	0.0308 ± 0.0021	17.7949 ± 0.1433	0.5442 ± 0.0073	1.0000 ± 0.0000
FedAvg	Inverting Gradients	0.0052 ± 0.0001	23.2985 ± 0.1220	0.7804 ± 0.0048	1.0000 ± 0.0000
FedPer($K_p = 1$)	DLG	0.1691 ± 0.0096	9.2412 ± 0.2896	0.0944 ± 0.0093	0.9600 ± 0.0000
FedPer($K_p = 1$)	Inverting Gradients	0.2253 ± 0.0024	7.3440 ± 0.0732	0.0820 ± 0.0055	0.6267 ± 0.0115
FedPer($K_p = 2$)	DLG	0.1747 ± 0.0046	9.0033 ± 0.2705	0.1035 ± 0.0146	0.8000 ± 0.0000
FedPer($K_p = 2$)	Inverting Gradients	0.1938 ± 0.0032	7.6631 ± 0.0887	0.1227 ± 0.0025	0.3667 ± 0.0115
FedPer($K_p = 3$)	DLG	0.1664 ± 0.0131	9.4759 ± 0.2471	0.1247 ± 0.0067	0.7867 ± 0.0115
FedPer($K_p = 3$)	Inverting Gradients	0.1810 ± 0.0013	7.9195 ± 0.0751	0.1405 ± 0.0034	0.3333 ± 0.0231
FedPer($K_p = 4$)	DLG	0.2115 ± 0.0062	7.8889 ± 0.2182	0.1305 ± 0.0115	0.7733 ± 0.0306
FedPer($K_p = 4$)	Inverting Gradients	0.1967 ± 0.0017	7.6120 ± 0.0893	0.1356 ± 0.0040	0.3800 ± 0.0000
FedPer($K_p = 5$)	DLG	0.1986 ± 0.0066	7.9210 ± 0.1649	0.0808 ± 0.0085	0.6267 ± 0.0306
FedPer($K_p = 5$)	Inverting Gradients	0.1443 ± 0.0028	8.5906 ± 0.0683	0.0982 ± 0.0011	0.6600 ± 0.0400
Per-FedAvg(FO)	DLG	0.0952 ± 0.0073	10.8221 ± 0.1479	0.0533 ± 0.0018	0.5400 ± 0.0000
Per-FedAvg(FO)	Inverting Gradients	0.2981 ± 0.0028	5.2771 ± 0.0364	0.0096 ± 0.0059	1.0000 ± 0.0000
Per-FedAvg(HF)	DLG	0.0925 ± 0.0053	10.8966 ± 0.0943	0.0534 ± 0.0021	0.2000 ± 0.0000
Per-FedAvg(HF)	Inverting Gradients	0.2988 ± 0.0031	5.2696 ± 0.0411	0.0062 ± 0.0054	0.9933 ± 0.0115
Per-FedAvg(HVP)	DLG	0.0891 ± 0.0049	10.9763 ± 0.1369	0.0544 ± 0.0008	0.2000 ± 0.0000
Per-FedAvg(HVP)	Inverting Gradients	0.2988 ± 0.0031	5.2698 ± 0.0405	0.0062 ± 0.0054	0.9933 ± 0.0115

Table 4.1 summarizes the average reconstruction performance for each evaluated algorithm/attack pair. The table includes FedAvg, FedPer with different numbers of personalized layers, and the three Per-FedAvg variants. For each algorithm, the table reports results for DLG and Inverting Gradients. The metric values are averaged over all communication rounds and all experimental runs. The uncertainty term shown after the $\pm$ symbol is the standard deviation computed over the run-level means.

FedAvg Results

Figures A.1–A.4 show the reconstruction metrics for FedAvg across communication rounds. These plots show how the attack performance changes during training for both DLG and Inverting Gradients.

Figure A.1 reports label recovery for FedAvg across communication rounds. The plot shows the proportion of correctly recovered labels at each communication round.

Figure A.2 reports MSE for FedAvg across communication rounds. This plot shows the pixel-level reconstruction error between reconstructed and original images at each communication round.

Figure A.3 reports PSNR for FedAvg across communication rounds. This plot shows the reconstruction fidelity at each communication round using a signal-to-noise-based image quality metric.

Figure A.4 reports SSIM for FedAvg across communication rounds. This plot shows the structural similarity between reconstructed and original images at each communication round.

FedPer Results

Figures A.5–A.8 show the reconstruction metrics for FedPer across communication rounds. The FedPer experiments include multiple personalization depths, denoted by K_p . These figures therefore show how reconstruction metrics vary across communication rounds for the different FedPer configurations and attack methods.

Figure A.5 reports label recovery for FedPer across communication rounds. The plot shows the proportion of correctly recovered labels for the evaluated FedPer configurations.

Figure A.6 reports MSE for FedPer across communication rounds. The plot shows the reconstruction error for each evaluated FedPer configuration and attack.

Figure A.7 reports PSNR for FedPer across communication rounds. The plot shows reconstruction fidelity for each FedPer configuration and attack method.

Figure A.8 reports SSIM for FedPer across communication rounds. The plot shows the structural similarity of the reconstructed images for the evaluated FedPer settings.

Per-FedAvg Results

Figures A.9–A.12 show the reconstruction metrics for the Per-FedAvg variants across communication rounds. The evaluated variants are the first-order approximation, the Hessian-free approximation, and the Hessian-vector product version.

Figure A.9 reports label recovery for the Per-FedAvg variants across communication rounds. The plot shows the proportion of labels recovered correctly by each attack method.

Figure A.10 reports MSE for the Per-FedAvg variants across communication rounds. The plot shows pixel-level reconstruction error for each variant and attack method.

Figure A.11 reports PSNR for the Per-FedAvg variants across communication rounds. The plot shows signal-fidelity-based reconstruction quality for each variant and attack method.

Figure A.12 reports SSIM for the Per-FedAvg variants across communication rounds. The plot shows structural similarity between reconstructed and original images for each Per-FedAvg configuration and attack method.

Metric Summary Figures

Figures 4.1–4.3 provide aggregate visual summaries of the reconstruction quality metrics. These figures correspond to the same averaged results reported in Table 4.1, but display the metric values graphically for easier comparison across algorithm/attack pairs.

Figure 4.1 summarizes the mean MSE values. The figure shows the average pixel-level reconstruction error for each evaluated algorithm/attack pair.

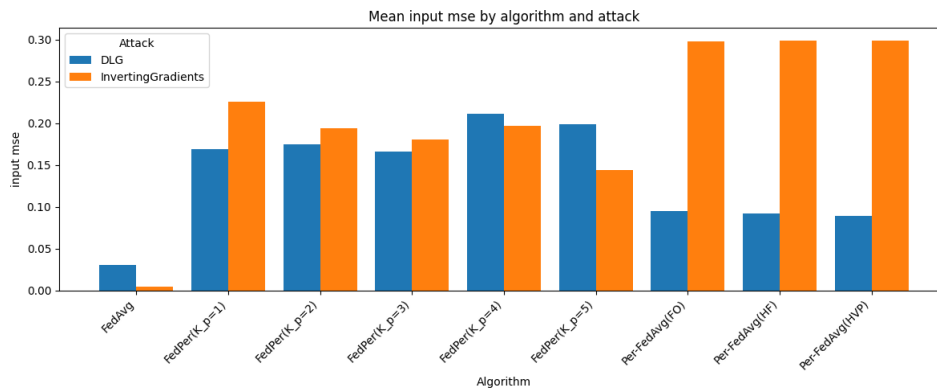


Figure 4.1: Mean MSE summary across evaluated algorithm/attack pairs.

Figure 4.2 summarizes the mean PSNR values. The figure shows the average signal-fidelity-based reconstruction quality for each evaluated algorithm/attack pair.

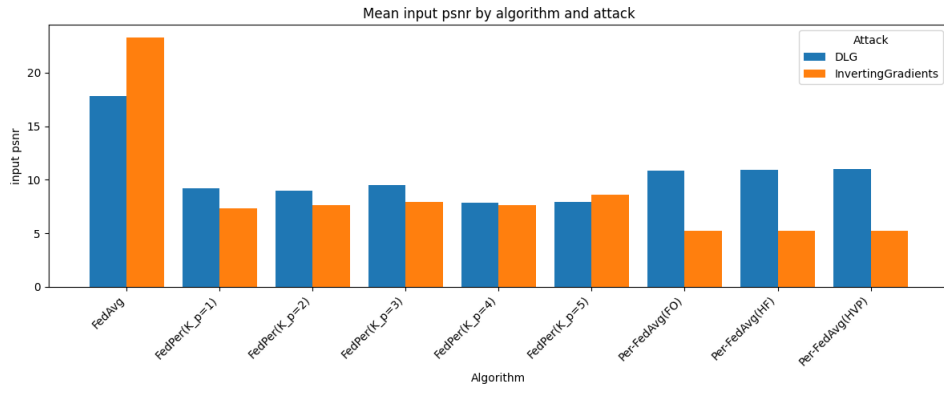


Figure 4.2: Mean PSNR summary across evaluated algorithm/attack pairs .

Figure 4.3 summarizes the mean SSIM values. The figure shows the average structural similarity between reconstructed and original images for each evaluated algorithm/attack pair.

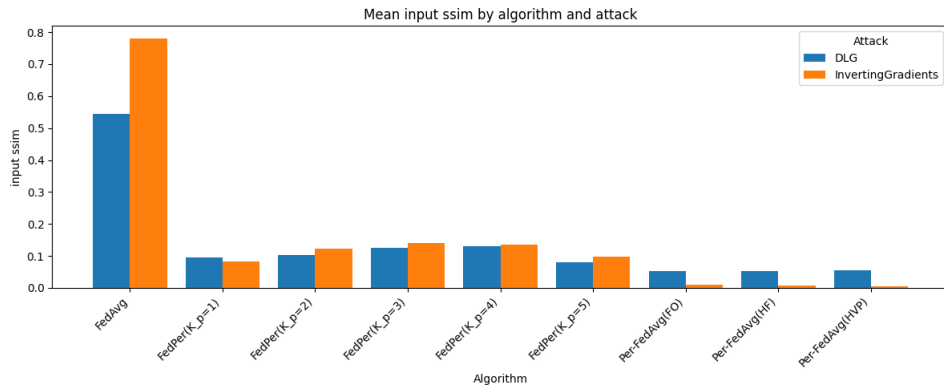


Figure 4.3: Mean SSIM summary across evaluated algorithm/attack pairs .

Figure 4.4 summarizes the mean label recovery values. The figure shows the average proportion of correctly recovered labels for each evaluated algorithm/attack pair.

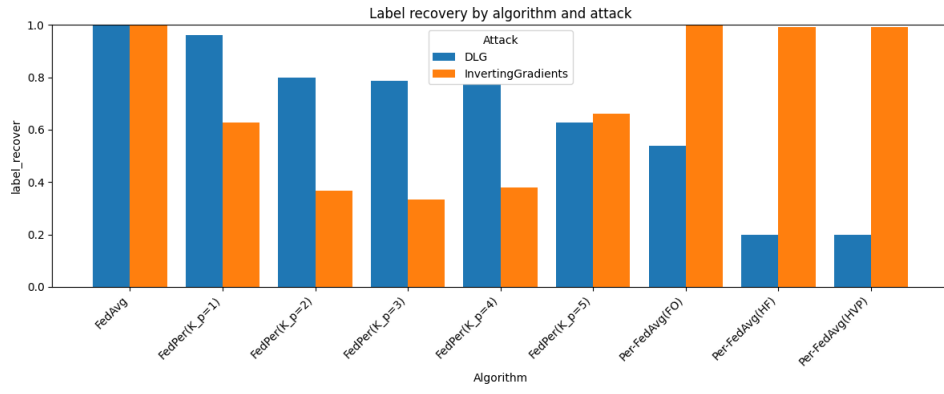


Figure 4.4: Mean Label Leakage summary across evaluated algorithm/attack pairs .

Overall, this chapter reports the reconstruction and label recovery results for all evaluated algorithm/attack pairs. The table provides the aggregate numerical results, while the figures show both communication-round trends and metric-level summaries. The interpretation of these results is reserved for the discussion chapter.

Experimental Scope and Interpretation

Before analyzing the individual results, it is necessary to contextualize the experimental objectives and the scope within which the results should be interpreted. The primary aim of this experiment was not to evaluate the absolute performance of gradient inversion attacks under realistic, production-grade federated learning conditions, but rather to assess the relative vulnerability of PFL algorithms compared to the canonical FedAvg baseline [McM+17]. Specifically, the adversarial models employed (DLG [ZLH19] and InvertingGradients (IG) [Gei+20]) were implemented without any modifications to account for the structural specifics of the PFL algorithms under evaluation. The attacks were optimized for the FedAvg threat model and applied, without adaptation, to FedPer and Per-FedAvg. Accordingly, all results should be interpreted as a measure of how structurally differing PFL training objectives and gradient formulations affect the effectiveness of traditional gradient inversion attacks.

Furthermore, the experimental setup employs single-sample-per-round client updates across a highly heterogeneous, two-class non-IID data partition. This configuration deliberately maximizes the potential for gradient leakage against FedAvg, since single sample updates produce gradients that strongly encode both image content and class information [ZLH19]. This ensures that any difference in attack effectiveness observed for PFL methods reflects genuine structural differences between the algorithms rather than an insufficient gradient signal in the baseline.

FedAvg Vulnerability to Gradient Inversion

The results for the FedAvg baseline confirm the established findings in the literature [ZLH19; Gei+20]. Standard federated averaging with full-model gradient sharing is highly susceptible to gradient inversion under single sample update conditions. Both DLG and IG achieve perfect label recovery (100%) over all 150 attack trials per attack variant, and both produce low reconstruction errors. IG substantially outperforms DLG on all image quality metrics, attaining a mean MSE of 0.0052, a mean SSIM of 0.780, and a mean PSNR of 23.3, compared to mean MSE of 0.0308, mean SSIM of 0.544, and mean PSNR of 17.8 for DLG. This is consistent with prior work demonstrating that cosine similarity based gradient matching provides a more reliable reconstruction signal than squared MSE gradient matching [Gei+20]. The strong label recovery observed in the FedAvg case is analytically expected. Because FedAvg transmits the full model update from a single training sample, the final-layer gradient is effectively unobscured, making label recovery trivial for both attacks. These results thus serve as the expected upper bound of attack effectiveness and validate the experimental setup prior to the evaluation of PFL algorithms.

Partial Privacy Through Layer Hiding

Image Reconstruction

FedPer [Ari+19] introduces a partition of the model into K_p personalized local layers and a set of shared layers whose gradients are transmitted to the server. The round-level trends (Figures A.6–A.8) reveal an important temporal structure in how this partitioning affects reconstruction quality. At communication round 1, before any personalization has accumulated, image reconstruction fidelity is high across

all FedPer variants for $K_p = 1$ through $K_p = 4$: both DLG and IG achieve MSE values below 0.05 for these configurations, placing round 1 performance close to the FedAvg baseline. The $K_p = 5$ variant is already performing poorly at round 1, consistent with the fact that hiding five of six trainable layers leaves the attacker with only a single shared layer’s gradient signal, which is insufficient for reliable image reconstruction even before personalization divergence sets in.

From round 2 onward, a large and sustained drop in reconstruction fidelity is visible for $K_p = 1$ through $K_p = 4$ across all image quality metrics. By round 10, even the most lightly personalized variant ($K_p = 1$ with DLG) degrades to $\text{SSIM} = 0.020$, far below its round 1 value. The $K_p = 5$ variant does not exhibit this same sharp drop, as its reconstruction was already at the noise floor from the first round. Aggregated across all rounds, mean MSE ranges from 0.1443 ($K_p = 5/\text{IG}$) to 0.2115 ($K_p = 4/\text{DLG}$), and mean SSIM ranges from 0.0808 ($K_p = 5/\text{DLG}$) to 0.1405 ($K_p = 3/\text{IG}$), indicating largely uninformative pixel-level reconstructions overall.

Two possible mechanisms account for this pattern. First, restricting the number of shared layers reduces the gradient information available to the attacker, shrinking the effective attack surface as K_p increases. This mechanism is primarily responsible for the poor round 1 performance of $K_p = 5$. Second, and the dominant driver for $K_p = 1$ through $K_p = 4$, the shared-layer gradients are computed through the client’s personalized local head w_{p_i} , whereas the attack reconstructs the dummy gradient through the global model head w_p . As local personalization layers diverge from the global initialization over successive communication rounds, this model mismatch grows, and the dummy gradient computed by the attacker becomes an increasingly poor approximation of the true client gradient. This explains why the sharp performance drop appears after round 1 and worsens progressively. Taken together, these observations suggest that the privacy benefit of FedPer’s layer-hiding mechanism operates primarily through two distinct effects: the reduction in visible gradient components (relevant mainly at higher K_p) and the accumulated personalized head mismatch (the dominant factor for lower K_p across rounds).

Label Recovery and Non-Monotonic Behavior

The round-level label recovery trends reveal a pattern analogous to that observed for image reconstruction. At communication round 1, before personalization has accumulated, all FedPer variant and attack combinations produce label recovery accuracy of 1.0, matching the FedAvg baseline. After round 1, label recovery becomes substantially more variable, and no clear monotonic or directional trend across communication rounds is visible in Figure A.5; the per-round values fluctuate without a consistent trajectory for any configuration.

When aggregated across all rounds, the label recovery summary (Figure 4.4) shows distinct patterns between the two attacks. DLG exhibits a roughly monotonic decline in label recovery as K_p increases, with rates of 0.960, 0.800, 0.787, 0.773, and 0.627 for $K_p = 1$ through 5 respectively. IG shows a qualitatively different and non-monotonic pattern: label recovery rates of 0.627, 0.367, 0.333, 0.380, and 0.660 across the same K_p values. Rather than decreasing monotonically with the number of personalized layers, IG accuracy falls from $K_p = 1$ through $K_p = 3$, then rises again at $K_p = 4$ and further at $K_p = 5$.

An explanation for the IG non-monotonicity could be that hiding more intermediate layers does not straightforwardly remove the directional class signal from the transmitted shared-layer gradients; rather, it alters the gradient space in ways that, at certain depths, cause the attack’s optimization to converge to a bad solution. By contrast, when sufficient intermediate features are removed ($K_p = 5$), the residual gradient direction appears to re-align with true class information, allowing partial label recovery. It should be emphasized that a definitive causal account of this pattern cannot be established from the current results alone. The non-monotonicity may partly arise from random noise across the limited three-run experimental design, and a dedicated experiment would be required to distinguish these explanations informatively.

Critically, even at its lowest point, IG label recovery (0.333 at $K_p = 3$) remains substantially above the random-guess baseline of 0.1 for a ten-class classification problem. DLG achieves well above this threshold across all K_p values, confirming that label leakage remains a meaningful risk across all FedPer configurations. The key implication is that increasing K_p does not monotonically improve privacy. FedPer changes the channel through which leakage occurs, reducing image-level leakage while transforming label leakage in a complex, non-linear fashion, rather than simply reducing all forms of information exposure uniformly.

Meta-Gradient Mismatch and Label-Image Divergence

Image Reconstruction

Per-FedAvg [FMO20] frames the federated optimization objective: rather than seeking a global model that performs well directly on all client distributions, it seeks a globally optimal initialization from which each client can rapidly adapt to its local personalized task using a small number of gradient steps. For Per-FedAvg, the meta-update incorporates an additional inner adaptation step and, in the second-order variants, a Hessian-vector product, producing an update of fundamentally different form than that of FedAvg. This mismatch severely disrupts image reconstruction for both attacks.

Across all communication rounds (Figures A.10–A.12), neither attack shows any identifiable declining or improving trend in image reconstruction quality as training progresses. Unlike FedPer, where a sharp drop from round 1 reflects the accumulation of personalization divergence, Per-FedAvg reconstruction quality is stable throughout training, remaining in a consistently poor region for all variants. This stability indicates that the reconstruction failure is a structural consequence of the meta-gradient formulation itself rather than an artifact of parameter drift over time.

In terms of relative attack performance, DLG consistently achieves lower MSE and comparable or higher SSIM than IG across all Per-FedAvg variants throughout training, which is the reverse of the FedAvg case. Aggregated across rounds, IG achieves extremely poor image fidelity: mean MSE of 0.2981 (FO), 0.2988 (HF), and 0.2988 (HVP), with mean SSIM values as low as 0.010 and 0.006 respectively, indicating reconstructions that are essentially uninformative. DLG achieves somewhat lower MSE values of 0.0952, 0.0925, and 0.0891 for FO, HF, and HVP, but with correspondingly very low SSIM values of approximately 0.053–0.054, also indicating visually meaningless reconstructions. Under the experimental settings and controls, image reconstruction of Per-FedAvg client data is therefore not feasible for either attack, with both remaining far below the quality achieved against FedAvg.

Label Recovery and the Cosine-Direction Effect

Despite the complete failure of both attacks to reconstruct input images, label leakage behaves very differently. IG achieves near perfect label recovery across all Per-FedAvg variants and throughout all communication rounds (Figure A.9): 1.000 for FO, 0.993 for HF, and 0.993 for HVP. All three variants exceed 0.99 label recovery, representing a level of leakage comparable to the FedAvg baseline despite producing completely uninformative image reconstructions. DLG, by contrast, recovers labels at rates of only 0.540 for FO, 0.200 for HF, and 0.200 for HVP.

The divergence between DLG and IG can be attributed to their respective optimization objectives. DLG minimizes the MSE between dummy and observed gradients, making it sensitive to the exact magnitude and component-wise structure of the gradient. When the observed update is not a simple single-step gradient, but instead a Per-FedAvg meta-update, the gradient-matching problem becomes misspecified. The attack is therefore forced to match a signal that no longer corresponds exactly to the gradient of one standard supervised training step. IG, by contrast, minimizes a cosine-based objective and therefore emphasizes the direction of the gradient rather than its absolute magnitude. This makes IG less sensitive to the scale changes introduced by the Per-FedAvg update and more sensitive to

whether the class direction of the update is preserved. This distinction is important for interpreting the high label recovery under Per-FedAvg. In classification with cross-entropy loss and one-hot labels, label information is strongly reflected in the output-layer gradient. In particular, the gradient associated with the correct class has a sign pattern that differs from the gradients associated with the incorrect classes, making the label signal primarily directional rather than purely magnitude-based [ZMB20]. The present results suggest that this directional label signal remains largely intact in the Per-FedAvg updates observed by the attacker. Therefore, even though the update is structurally different from a standard FedAvg gradient, it still appears to point in a sufficiently similar class discriminative direction for IG to identify the correct label.

For the First-Order Per-FedAvg variant, this explanation is relatively direct. FO-MAML drops the second-order Hessian correction and uses the gradient after the inner adaptation step. Under the experimental setting used here, where the inner and outer learning rates are equal and the same data sample is used for both adaptation and meta-update, the FO update remains close in form to a short sequence of local gradient descent steps. This makes it plausible that the class-related direction of the original supervised gradient is preserved strongly enough for IG to recover the label. The HF and HVP variants require a more careful interpretation. Their meta-updates contain second-order information through a Hessian-vector correction. Depending on the local curvature of the loss surface and the size of the adaptation step, the Hessian correction may preserve, distort, dampen, or even reverse parts of the first-order direction. The fact that IG still achieves near-perfect label recovery for HF and HVP therefore suggests that, in this experimental setting, the second-order correction did not dominate or sufficiently deviate the first-order class signal. A likely contributing factor is that the same sample was used for the inner adaptation and the outer meta-gradient computation. This differs from many meta-learning implementations, where adaptation and meta-objective gradients are computed on separate batches. Using the same sample can make the adapted-gradient direction remain strongly tied to the original sample and label, thereby reducing the directional separation between the first-order gradient and the resulting meta-update.

Consequently, the high IG label recovery should not be interpreted as evidence that Per-FedAvg meta-gradients always preserve labels, or that second-order corrections are harmless from a privacy perspective. Rather, it shows that, under the chosen step sizes, single sample batch setting, and same-sample adaptation/meta-update protocol, the meta-update still preserves enough of the label-informative gradient direction for a cosine-based inversion attack to recover the label almost perfectly. The image reconstruction failure and label recovery success therefore reveal two different leakage channels: the pixel-level reconstruction signal is heavily degraded, while the output-layer class signal remains highly exposed.

Leakage Channels Are Not Interchangeable

A central implication of these results is that image reconstruction quality and label recovery are not interchangeable measures of privacy. In the Per-FedAvg setting, IG achieves mean SSIM values as low as 0.006 while simultaneously recovering labels with 99.3% accuracy. The worst individual reconstructions by MSE in the entire dataset include several Per-FedAvg cases where the true label was nonetheless recovered correctly. This clearly demonstrates that an attack can fail entirely at pixel-level reconstruction while still extracting highly sensitive semantic information in the form of the training label. Label leakage can be severe even when pixel-level reconstruction is negligible. Given that training labels often encode sensitive private attributes (such as disease diagnoses or user behavior categories), this residual leakage channel constitutes a meaningful privacy risk that should not be dismissed.

These findings suggest that personalization, as implemented in both FedPer and Per-FedAvg, transforms the form of gradient leakage leading to pixel-level privacy improvements, while label-level privacy remains a significant vulnerability (still comparatively improved). The experiment directly demon-

strates how FedAvg, FedPer, and Per-FedAvg behave under a common controlled gradient inversion setup, showing how changes to the server visible update affect the ability of DLG and InvertingGradients to reconstruct private training images and labels. It can be cautiously inferred that personalization changes the leakage channel (FedPer removes certain gradient components from the attacker and causes a mismatch between the client and server models, while Per-FedAvg changes the update objective itself). Which, under the experimental constraints and control settings, showed that personalization improves privacy than standard federated learning and reduces the effectiveness of the GIA attacks. However it does not support production level vulnerability claims nor does it mean that PFL algorithms explored in this thesis are invulnerable to privacy attacks and information leakage (as label leakage is quite prevalent). The main contribution is therefore an exploratory baseline: it demonstrates that the structural form of personalization matters for gradient inversion and identifies several directions for more complete empirical investigations.

Limitations and Future Work

The experiment was designed as a controlled comparison between FedAvg, FedPer, and Per-FedAvg variants under the same model architecture, data split, client setup, and attack configuration. This makes the results useful for analyzing how the algorithmic structure of the transmitted update affects reconstruction quality. However, the experiment should not be interpreted as a complete privacy audit of personalized federated learning systems in general. It is instead a simplified baseline experiment in which many factors were fixed in order to isolate the effect of the learning algorithm itself. This distinction is important because practical federated learning systems usually involve additional factors such as multiple local steps, mini-batches, client aggregation, and communication constraints, all of which can affect both training behavior and leakage risk [McM+17].

Limitations

A first limitation concerns the finite-difference parameter δ used in the Hessian-free Per-FedAvg variant. In the current implementation, δ is fixed to 10^{-2} and is used to approximate the Hessian-vector product as

$$H_i v_i \approx \frac{\nabla f_i(\theta + \delta v_i) - \nabla f_i(\theta - \delta v_i)}{2\delta}.$$

This approximation is not merely an implementation detail, because it enters directly into the Per-FedAvg meta-update and therefore affects the client update observed by the attacker. Hessian-free MAML-style methods are intended to approximate second-order behavior without explicitly computing the full Hessian, so the quality of this approximation matters for the intended algorithmic behavior and the resulting leakage signal [fallah2019convergence]. Since finite-difference approximations depend on the perturbation size, a poorly chosen δ may produce a Hessian-vector approximation that is numerically unstable, too small, or dominated by approximation error [finite_difference]. This may influence the structure of the observed update and, consequently, the effectiveness of the reconstruction attack.

A second limitation is the model architecture. The experiment used a LeNet-style CNN with sigmoid activations in the hidden layers. This architecture was computationally light and therefore suitable for repeated gradient inversion experiments. However, activation functions affect the gradient signal available to the attacker. Sigmoid activations can saturate, which may reduce gradient magnitudes and weaken the optimization signal in parts of the network. ReLU activations can reduce some saturation effects, although they introduce their own issues through inactive neurons. Since the experiment showed that changing the visible gradient information affects reconstruction quality, the activation function may also affect how much information is preserved in the gradients. If an activation function produces stronger or less saturated gradients, it may make gradient inversion attacks more effective. Conversely, weaker or more saturated gradients may reduce reconstruction quality independently of the federated

learning algorithm being evaluated. This architectural limitation is especially relevant for the FedPer results. FedPer changes the attack surface by hiding the final personalized layers from the server. As K_p increases, fewer trainable layers are visible to the attacker. The attack therefore has access to a smaller part of the full gradient signal. However, the amount of information contained in the remaining shared layers may still depend on the model architecture and activation functions. If the visible layers preserve strong class-discriminative gradients, then even partial updates may remain informative. Conversely, if the visible layers produce weak or saturated gradients, the attack may struggle even when several layers are shared.

A third limitation concerns attack hyperparameter tuning. The attack settings were tuned on a single CIFAR-10 image in a simple FedAvg reconstruction setting. This made the tuning procedure computationally feasible, but it also means that the selected settings may be partly fitted to that specific image. A stronger tuning procedure would use a calibration set containing multiple images from all CIFAR-10 classes and report the mean and variance of reconstruction metrics. This would reduce the risk that the selected attack parameters perform well only for one image or one class. Since the main experiment evaluates reconstruction across clients, rounds, algorithms, and attacks, the attack calibration should ideally be based on a more representative sample of images. In addition, the hyperparameter search selected the best settings using MSE as the primary criterion. This is reasonable because MSE directly measures pixel-wise reconstruction error and is also used in the main experiment. However, MSE does not fully capture perceptual reconstruction quality. In hindsight, the selection criterion could have used a combined score involving MSE, PSNR, and SSIM. This would have made the tuning procedure more aligned with the final interpretation, where all three reconstruction metrics are considered together. Minimal MSE does not always imply maximal SSIM, even though the two are often correlated. A combined score could therefore have selected attack settings that were better balanced between pixel accuracy and structural similarity.

A fourth limitation is that the attack hyperparameters were tuned on the FedAvg baseline and then reused across all FedAvg, FedPer, and Per-FedAvg configurations. This was an intentional design choice. The goal was to compare how the same calibrated attacks behave when the visible update structure changes from a full FedAvg update, to a partial FedPer update, to a Per-FedAvg meta-update. This supports the main research objective because it isolates the structural effect of the algorithm under fixed attack settings. However, it does not answer how vulnerable each algorithm is against its individually strongest possible attack. FedPer partial updates and Per-FedAvg meta-updates may have different optimization landscapes from FedAvg updates. Therefore, attack hyperparameters tuned on FedAvg may not be optimal for these personalized settings. The current experiment can therefore support statements about relative leakage under a common FedAvg-calibrated attack configuration. It can show whether algorithmic structure changes reconstruction quality when the attack is held fixed. It does not claim that a given algorithm is secure against gradient inversion in an absolute sense. To answer that stronger question, each algorithm would need its own attack tuning procedure. Such an experiment would be closer to a worst-case vulnerability evaluation, where the attacker is allowed to optimize its method separately for each FL or PFL algorithm.

A fifth limitation is that the experiment uses a batch size of one and one local optimization step per communication round. This is useful for isolating leakage from a single update, because the transmitted update corresponds directly to one private image. It also creates a strongly attacker favorable setting, since there is no gradient averaging across multiple samples and no smoothing from several local optimization steps. However, this does not reflect most realistic federated learning deployments. In practical FL settings, clients often train on mini-batches and perform several local steps before sending an update. FedAvg, for example, is commonly understood as allowing multiple local batch updates before communicating updated weights to the server rather than transmitting only a single gradient step [McM+17]. These processes mix information from multiple examples and may make exact reconstruction harder.

Finally, several extensions were excluded due to time and computational constraints. Gradient inversion attacks are expensive to run, especially when repeated across multiple algorithms, clients, communication rounds, and hyperparameter settings. The current experiment already contains 900 attack reconstruction records and saved image outputs. For this reason, the experiment prioritized the parameters and controls most directly connected to the research objective: the algorithmic structure of the visible update, the difference between full and partial model sharing, and the difference between ordinary gradient updates and Per-FedAvg meta-updates.

Future Work

Future work should first treat the Hessian-free finite-difference parameter δ as an explicit experimental hyperparameter rather than a fixed constant. Evaluating multiple values would make it possible to determine whether the observed behavior of the Hessian-free Per-FedAvg variant is caused by the intended second-order structure of the algorithm or by numerical artifacts introduced by the approximation itself. This is particularly important because changes in DLG or Inverting Gradients reconstruction quality may otherwise reflect the quality of the finite-difference estimate rather than the privacy properties of the Hessian-free algorithmic structure being studied.

A second direction is to repeat the experiment with additional activation functions and model architectures. Testing ReLU, tanh, and sigmoid variants would help determine whether the observed leakage patterns are stable across different gradient regimes. This is especially relevant because gradient inversion attacks depend directly on the information contained in the transmitted gradients. If an architecture produces stronger, less saturated, or more class-informative gradients, then the privacy behavior observed in this thesis may change.

A larger model should also be used to investigate the FedPer K_p effect more thoroughly. The current LeNet-style architecture contains only a small number of trainable layers, so each increase in K_p removes a relatively large portion of the model from the attacker. This makes the layer-hiding effect coarse-grained. A deeper CNN would allow more fine-grained variation in the number and position of personalized layers. This would make it possible to test whether the non-monotonic rise in IG label recovery observed at higher K_p values is a stable pattern or a one-time artifact of the specific architecture and limited experimental setting. In particular, a larger model could reveal whether label recovery increases again after sufficient intermediate layers are hidden, or whether the observed rise at $K_p = 4$ and $K_p = 5$ disappears when the layer partition is evaluated at higher resolution.

Future work should also tune the attacks separately for each algorithm and configuration. The current experiment intentionally reuses FedAvg-calibrated attack settings in order to compare structural effects under a fixed attack. A stronger privacy evaluation would instead allow DLG and Inverting Gradients to be optimized separately for FedAvg, FedPer, and Per-FedAvg. This would help distinguish between two explanations for poor reconstruction: whether personalization genuinely reduces available leakage, or whether the attack hyperparameters are simply poorly matched to the personalized update structure.

Another important extension is to evaluate larger batch sizes, multiple local steps, and client aggregation. These settings would move the experiment closer to practical FL deployments, where updates often combine information from several samples and several local optimization steps. This would help determine whether the label leakage observed under single sample updates persists when gradients are averaged over multiple examples. It would also clarify whether the image-reconstruction failure observed for FedPer and Per-FedAvg remains stable under more realistic update conditions.

Future work should also evaluate the privacy/utility tradeoff directly. The current experiment focuses on reconstruction leakage and does not measure whether the tested configurations preserve useful predictive performance. A more complete evaluation should therefore report both attack metrics, such

as MSE, PSNR, SSIM, and label recovery, and utility metrics, such as test accuracy or client-specific personalized accuracy. This would make it possible to determine whether a configuration that reduces leakage also preserves the intended learning objective. This is particularly important for PFL, because personalization is usually motivated by improved performance under heterogeneous client data. Without utility measurements, it is not possible to determine whether a privacy improvement reflects a meaningful privacy/utility tradeoff or simply a degradation of the learning process itself.

Finally, future experiments could separate data into multiple batches in the Per-FedAvg adaptation and meta-update procedure. In the current implementation, the same sample is used for both the inner adaptation and the outer meta-gradient computation. This may preserve the label-informative direction more strongly than a conventional meta-learning setup where adaptation and evaluation are computed on distinct batches. Repeating the experiment with separate batches would help determine whether the near perfect IG label recovery in Per-FedAvg is caused by the Per-FedAvg structure itself, the same sample protocol, or a combination of both.

Conclusion

This thesis investigated how personalization strategies in federated learning influence the effectiveness of gradient inversion attacks compared to standard non-personalized federated learning. Through a controlled exploratory experiment, FedAvg, FedPer, and Per-FedAvg were evaluated under the same model architecture, data setup, client configuration, and attack settings using DLG and Inverting Gradients. The results show that personalization reduces information leakage in both label and image, but that does not mean that PFL algorithms are invulnerable to it, more specifically label leakage. FedAvg remained highly vulnerable in the single-sample setting, with both attacks recovering labels successfully and Inverting Gradients producing the strongest image reconstructions. FedPer reduced image reconstruction quality as more personalized layers were kept private, but label leakage still remained in several configurations, showing that shared representation layers can continue to expose class-related information. Per-FedAvg produced the strongest separation between image and label leakage: its meta-update structure generally prevented visually meaningful image reconstruction, while Inverting Gradients still recovered labels with high accuracy. These results indicate that personalization can reduce pixel-level reconstruction risk, but it should not be interpreted as a complete privacy defense. The thesis therefore answers the research question by showing that personalized federated learning does affect gradient inversion attack effectiveness, but the effect depends on both the personalization mechanism and the attack objective. The conclusions are limited by the controlled setup, including the single sample setting, fixed attack hyperparameters, limited communication rounds, and simplified Per-FedAvg implementation. Future work should evaluate larger batches, multiple local steps, deeper architectures, separately tuned attacks, and more to determine whether the observed reduction in image leakage and persistence of label leakage also hold in more realistic federated learning settings.

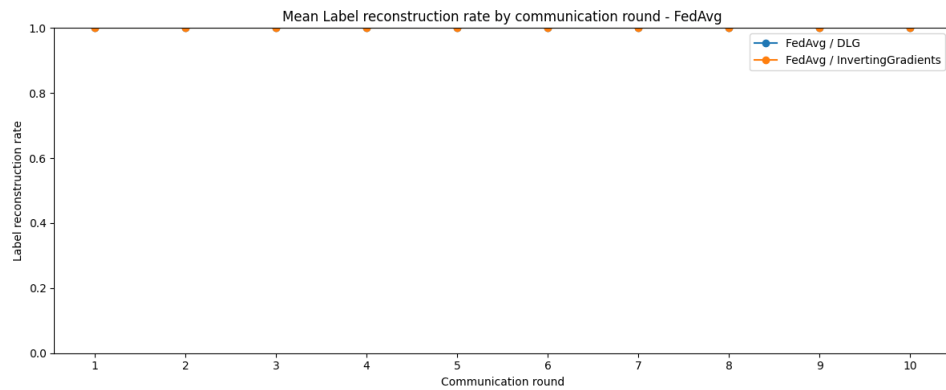


Figure A.1: FedAvg label recovery per communication round .

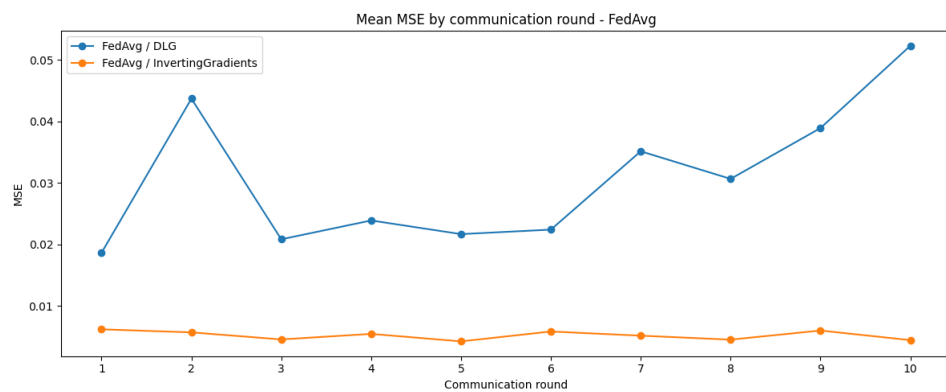


Figure A.2: FedAvg MSE per communication round .

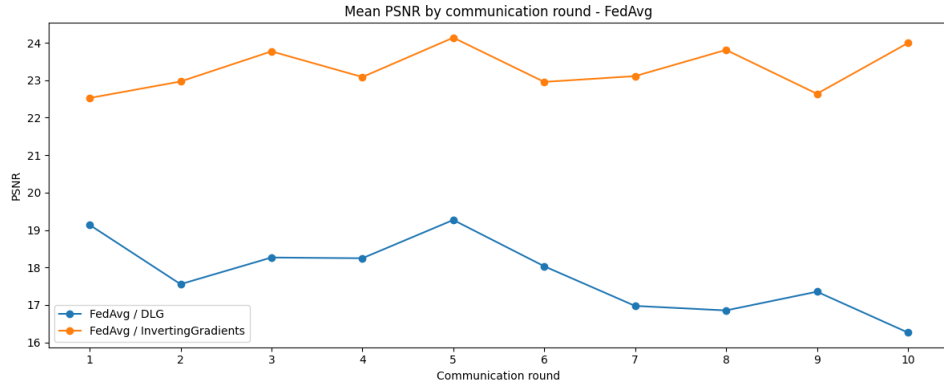


Figure A.3: FedAvg PSNR per communication round .

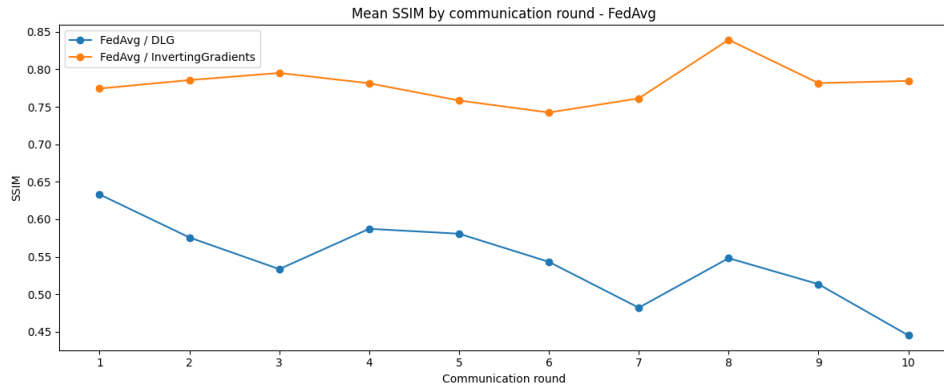


Figure A.4: FedAvg SSIM per communication round .

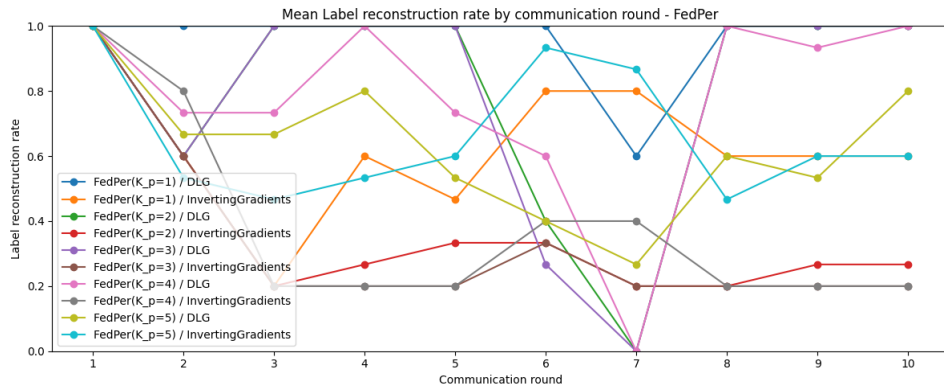


Figure A.5: FedPer label recovery per communication round .

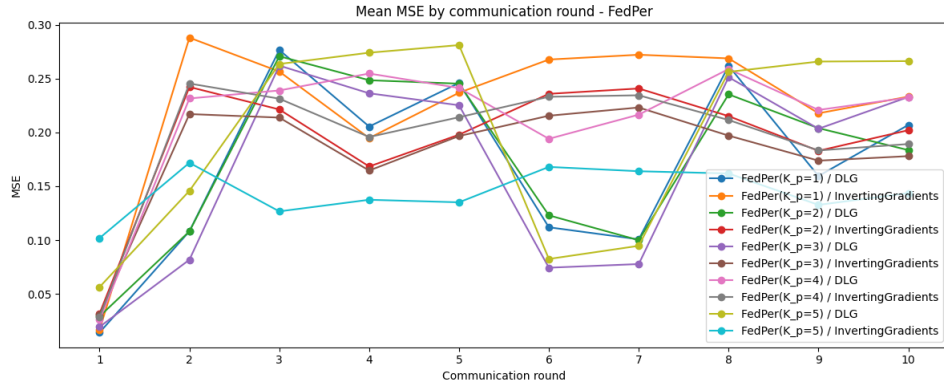


Figure A.6: FedPer MSE per communication round .

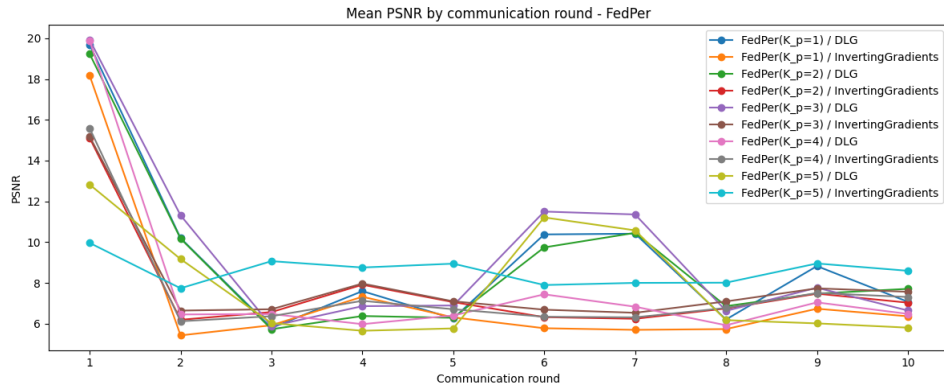


Figure A.7: FedPer PSNR per communication round .

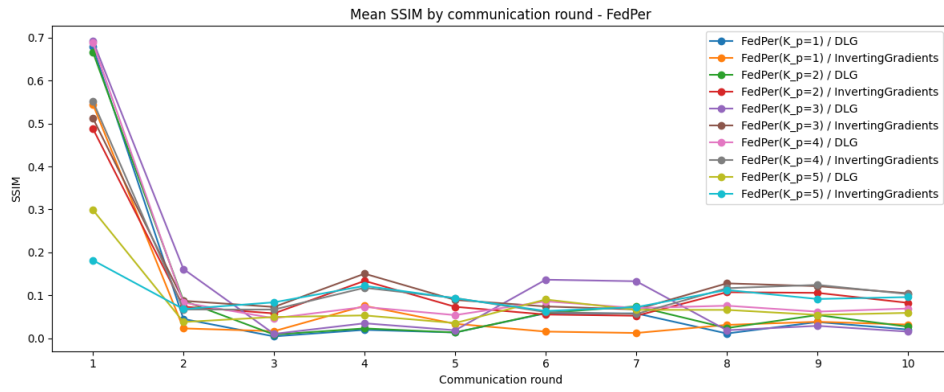


Figure A.8: FedPer SSIM per communication round .

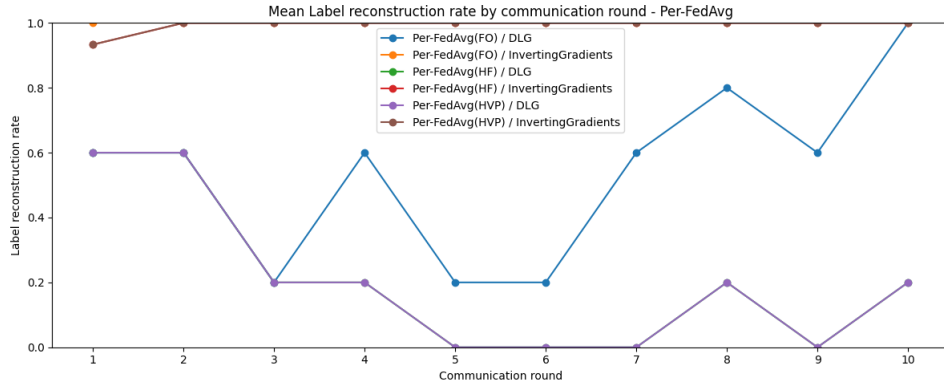


Figure A.9: Per-FedAvg label recovery per communication round .

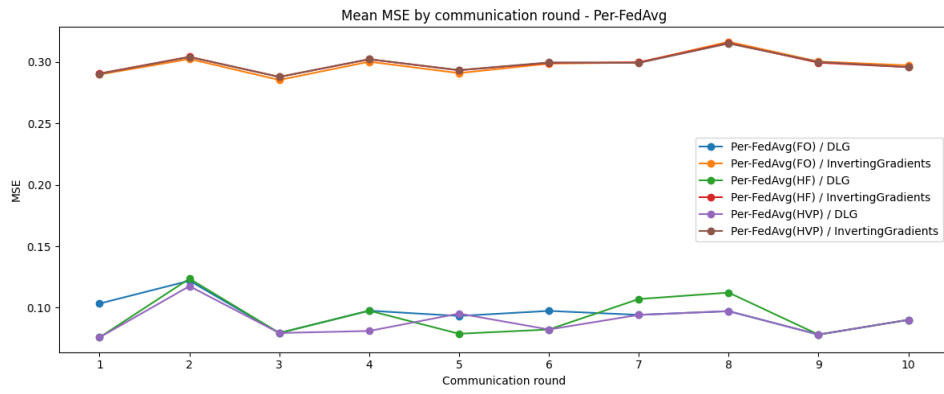


Figure A.10: Per-FedAvg MSE per communication round .

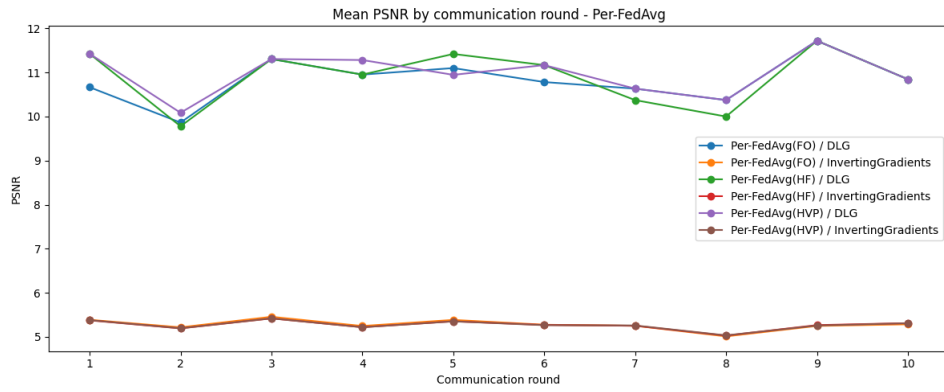


Figure A.11: Per-FedAvg PSNR per communication round .

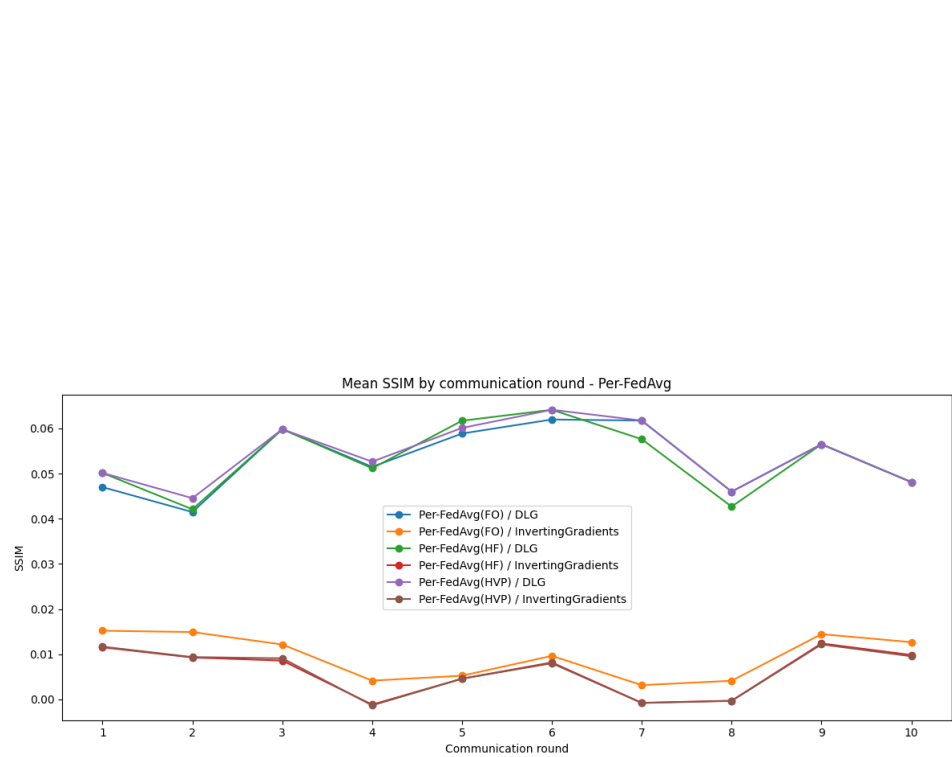


Figure A.12: Per-FedAvg SSIM per communication round .

Bibliography

- [Ari+19] Manoj Ghuhan Arivazhagan et al. *Federated Learning with Personalization Layers*. Preliminary work, under review by AISTATS 2020. 2019. arXiv: 1912.00818 [cs.LG].
- [Bho+18] Abhishek Bhowmick et al. *Protection Against Reconstruction and Its Applications in Private Federated Learning*. 2018. arXiv: 1812.00984 [stat.ML]. URL: <https://arxiv.org/abs/1812.00984>.
- [Dim+22] Dimitar Iliev Dimitrov et al. “Data Leakage in Federated Averaging.” In: *Transactions on Machine Learning Research* (2022). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=e7A0B99zJf>.
- [DTN20] Canh T. Dinh, Nguyen H. Tran, and Tuan Dung Nguyen. “Personalized Federated Learning with Moreau Envelopes.” In: *Advances in Neural Information Processing Systems*. Volume 33. 2020, pages 21394–21405.
- [FAL17] Chelsea Finn, Pieter Abbeel, and Sergey Levine. “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks.” In: *Proceedings of the 34th International Conference on Machine Learning*. Edited by Doina Precup and Yee Whye Teh. Volume 70. Proceedings of Machine Learning Research. PMLR, June 2017, pages 1126–1135. URL: <https://proceedings.mlr.press/v70/finn17a.html>.
- [FJR15] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. “Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures.” In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pages 1322–1333. DOI: 10.1145/2810103.2813677.
- [FMO20] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. “Personalized Federated Learning: A Meta-Learning Approach.” In: (February 2020). DOI: 10.48550/arXiv.2002.07948.
- [Gei+20] Jonas Geiping et al. “Inverting Gradients - How easy is it to break privacy in federated learning?” In: *CoRR* abs/2003.14053 (2020). arXiv: 2003.14053. URL: <https://arxiv.org/abs/2003.14053>.
- [Hua+21] Yangsibo Huang et al. “Evaluating Gradient Inversion Attacks and Defenses in Federated Learning.” In: *Advances in Neural Information Processing Systems*. Volume 34. 2021, pages 7232–7241.
- [HZ10] Alain Horé and Djemel Ziou. “Image Quality Metrics: PSNR vs. SSIM.” In: *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)*. IEEE, 2010, pages 2366–2369. DOI: 10.1109/ICPR.2010.579.
- [Kai+21] Peter Kairouz et al. “Advances and Open Problems in Federated Learning.” In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pages 1–210. DOI: 10.1561/22000000083.
- [Kar+20] Sai Praneeth Karimireddy et al. “SCAFFOLD: Stochastic Controlled Averaging for Federated Learning.” In: *Proceedings of the 37th International Conference on Machine Learning*. Volume 119. Proceedings of Machine Learning Research. PMLR, 2020, pages 5132–5143. URL: <https://proceedings.mlr.press/v119/karimireddy20a.html>.
- [Lec+98] Y. Lecun et al. “Gradient-based learning applied to document recognition.” In: *Proceedings of the IEEE* 86.11 (1998), pages 2278–2324. DOI: 10.1109/5.726791.

- [Li+20a] Tian Li et al. “Federated Optimization in Heterogeneous Networks.” In: *Proceedings of Machine Learning and Systems*. Volume 2. 2020, pages 429–450.
- [Li+20b] Xiang Li et al. “On the Convergence of FedAvg on Non-IID Data.” In: *International Conference on Learning Representations*. 2020. URL: <https://openreview.net/forum?id=HJxNAnVtDS>.
- [LYY20] Lingjuan Lyu, Han Yu, and Qiang Yang. *Threats to Federated Learning: A Survey*. 2020. arXiv: 2003.02133 [cs.LG]. URL: <https://arxiv.org/abs/2003.02133>.
- [Man+20] Yishay Mansour et al. “Three Approaches for Personalization with Applications to Federated Learning.” In: *arXiv preprint arXiv:2002.10619* (2020).
- [McM+17] Brendan McMahan et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data.” In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Edited by Aarti Singh and Jerry Zhu. Volume 54. Proceedings of Machine Learning Research. PMLR, 20–22 Apr 2017, pages 1273–1282. URL: <https://proceedings.mlr.press/v54/mcmahan17a.html>.
- [Mel+19] Luca Melis et al. “Exploiting Unintended Feature Leakage in Collaborative Learning.” In: *2019 IEEE Symposium on Security and Privacy*. IEEE, 2019, pages 691–706. DOI: 10.1109/SP.2019.00029.
- [Son+20] Xingyou Song et al. “ES-MAML: Simple Hessian-Free Meta Learning.” In: *International Conference on Learning Representations (ICLR)*. 2020. URL: <https://openreview.net/forum?id=S1exA2NtDB>.
- [Tan+23] Alysia Ziying Tan et al. “Towards Personalized Federated Learning.” In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (2023), pages 9587–9603. DOI: 10.1109/TNNLS.2022.3160699.
- [Zha+21] Chen Zhang et al. “A survey on federated learning.” In: *Knowledge-Based Systems* 216 (2021), page 106775.
- [ZLH19] Ligeng Zhu, Zhijian Liu, and Song Han. “Deep leakage from gradients.” In: *Advances in neural information processing systems* 32 (2019).
- [ZMB20] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. “iDLG: Improved Deep Leakage from Gradients.” In: *arXiv preprint arXiv:2001.02610* (2020). URL: <https://arxiv.org/abs/2001.02610>.